

1. Semana 1. Introducción al Modelado Científico

Objetivo de la cátedra

Comprender los fundamentos conceptuales del modelado científico y su importancia dentro de la investigación doctoral aplicada a ingeniería y tecnologías.

Temario

- Introducción a la simulación científica.
- Diferencia entre modelo físico, matemático y computacional.
- Abstracción y simplificación de fenómenos reales.
- Variables, parámetros y condiciones de frontera.
- Escalas espaciales y temporales.
- Error, precisión e incertidumbre.
- Reproducibilidad científica.
- Discusión de casos reales de investigación doctoral.

Proposito del tema. En esta semana se establece la base conceptual y matematica del curso. Se analiza la relacion entre fenomeno real, modelo fisico, modelo matematico, modelo computacional, simulacion, prototipo y validacion. El objetivo no consiste en aprender un software especifico, sino en comprender como se transforma un problema de investigacion en un sistema formal capaz de generar predicciones, estimaciones, escenarios y decisiones tecnicamente justificadas.

Resultado de aprendizaje esperado

Al finalizar la sesion, el estudiante sera capaz de formular un problema de investigacion como un modelo cientifico explicito, identificando sus variables de estado, parametros, condiciones iniciales, condiciones de frontera, hipotesis de simplificacion, fuentes de error, indicadores de validacion y requisitos minimos de reproducibilidad. Ademas, debera ser capaz de utilizar una herramienta de inteligencia artificial generativa como apoyo para construir, ejecutar y documentar una simulacion computacional simple, sin delegar el juicio cientifico al modelo generativo, porque la IA redacta codigo, no sustituye el criterio metodologico; seria pedirle brujula moral a una calculadora.

1.1. Motivacion doctoral: de la observacion al modelo

Una investigacion doctoral aplicada en ingenieria y tecnologia rara vez inicia con una ecuacion. Normalmente inicia con un fenomeno: un cable que se calienta bajo cortocircuito, una bomba centrifuga que produce hemolisis, una estructura que debe soportar

carga, una imagen medica que debe clasificarse, un sistema de trafico que debe controlarse o una cadena de decisiones humanas que debe representarse mediante agentes. La ecuacion aparece despues, cuando el fenomeno se abstraee en terminos de variables, restricciones, parametros y leyes de evolucion.

En terminos generales, se considera un sistema fisico, biologico, social o computacional definido sobre un dominio espacial $\Omega \subset \mathbb{R}^d$ y un intervalo temporal $t \in [0, T]$. El estado del sistema se representa mediante una funcion

$$\mathbf{u} : \Omega \times [0, T] \rightarrow \mathbb{R}^m, \quad (1)$$

donde $\mathbf{u}(\mathbf{x}, t)$ agrupa las variables relevantes del problema. Por ejemplo, en transferencia de calor, $\mathbf{u}$ puede ser la temperatura $T(\mathbf{x}, t)$; en mecanica estructural, el desplazamiento $\mathbf{u}(\mathbf{x}, t)$; en dinamica de fluidos, el campo de velocidades $\mathbf{v}(\mathbf{x}, t)$ y la presion $p(\mathbf{x}, t)$; en un sistema RAG, el estado computacional puede expresarse como un conjunto de documentos recuperados, vectores de representacion y distribuciones de probabilidad sobre secuencias de texto.

La estructura minima de un modelo cientifico puede escribirse como

$$\mathcal{M} : (\mathbf{x}, t; \boldsymbol{\theta}, \mathbf{c}) \mapsto \hat{\mathbf{u}}(\mathbf{x}, t), \quad (2)$$

donde $\boldsymbol{\theta}$ representa parametros del modelo, $\mathbf{c}$ representa controles, entradas o condiciones externas, y $\hat{\mathbf{u}}$ representa la prediccion del modelo. Una medicion experimental se expresa como

$$\mathbf{y} = \mathcal{H}(\mathbf{u}) + \boldsymbol{\varepsilon}, \quad (3)$$

donde $\mathcal{H}$ es el operador de observacion y $\boldsymbol{\varepsilon}$ representa ruido, sesgo, error de instrumentacion o incertidumbre experimental. Esta formulacion permite distinguir con claridad tres cosas que suelen mezclarse de manera peligrosa: el fenomeno real, el modelo que lo representa y el dato que se mide.

1.2. Cadena metodologica del curso

En este curso se adoptara la siguiente cadena:

$$\begin{aligned} \text{Fenomeno real} &\rightarrow \text{Modelo fisico} \rightarrow \text{Modelo matematico} \rightarrow \text{Modelo computacional} \\ &\rightarrow \text{Simulacion} \rightarrow \text{Prototipo} \rightarrow \text{Validacion} \end{aligned} \quad (4)$$

Esta cadena no es lineal en sentido rigido. En investigacion real, se avanza y se retrocede. Una simulacion puede revelar que una hipotesis fisica es falsa; un prototipo puede mostrar que una variable omitida domina el fenomeno; una validacion experimental puede obligar a reformular el modelo. La madurez doctoral consiste precisamente en

documentar ese ciclo sin fingir que todo salio perfecto desde el primer intento, una fantasia que solo sobrevive en informes administrativos y cuentos infantiles. La estructura conceptual general del curso se resume en la [Figura 1](#), donde se establece la transicion entre fenomeno real, formulacion matematica, simulacion computacional, prototipado y validacion experimental.

1.3. Problema bien planteado y problema inverso

Un modelo no se considera utilizable solo porque tenga ecuaciones. En el sentido de Hadamard, un problema esta *bien planteado* (well-posed) si cumple tres condiciones:

1. **Existencia.** Existe una solucion u para los datos (θ, c, u_0, g) admisibles.
2. **Unicidad.** La solucion es unica en el espacio funcional considerado.
3. **Estabilidad continua.** La solucion depende continuamente de los datos:

$$\|u_1 - u_2\| \leq C \|d_1 - d_2\|, \quad (5)$$

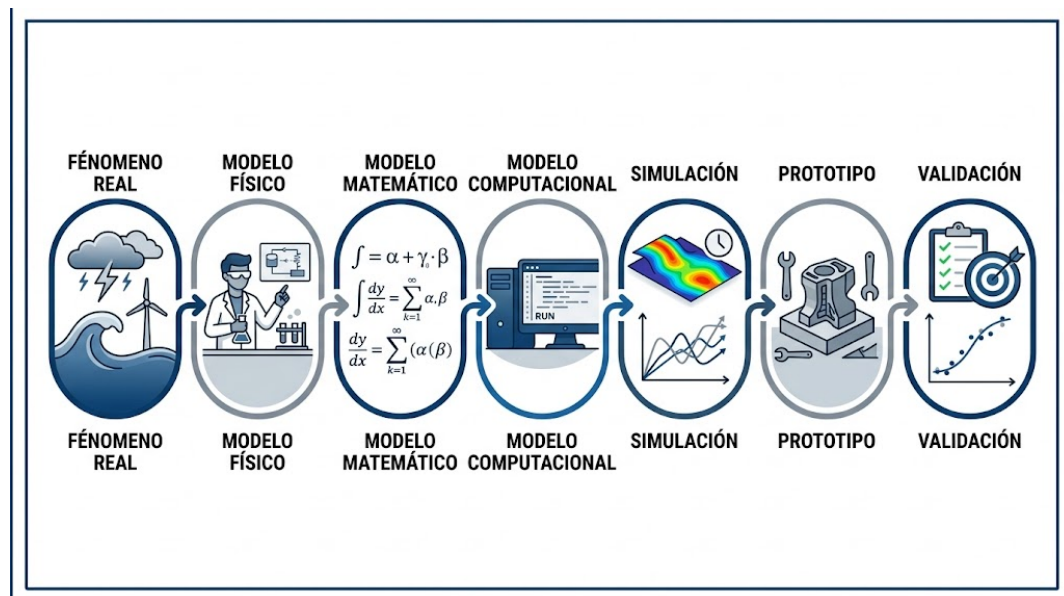
donde d agrupa los datos del problema y C es independiente de la solucion.

La falla del tercer requisito es la mas peligrosa, porque produce modelos que “funcionan” hasta que un dato cambia ligeramente. En problemas *directos* (forward problems) suele cumplirse, pero en problemas *inversos* (estimar θ a partir de y) la estabilidad rara vez es automatica y se requiere regularizacion: Tikhonov, restricciones bayesianas o priors fisicos. Buena parte de la metrologia, calibracion experimental y data assimilation cae en este regimen.

Un concepto relacionado es la *identifiabilidad* estructural y practica de los parametros: aunque el problema sea bien planteado para u , distintos θ pueden producir $\hat{u}$ indistinguibles dentro del ruido ϵ . Antes de invertir esfuerzo computacional en calibracion, se debe verificar que los datos disponibles tienen sensibilidad suficiente respecto a los parametros que se buscan estimar.

Tabla 1: Correspondencia entre niveles de representacion cientifica.

Nivel	Pregunta central	Ejemplo
Fenomeno real	?Que ocurre en el sistema observado?	Un cable OPGW se calienta durante un cortocircuito.
Modelo fisico	?Que mecanismos dominan el comportamiento?	Efecto Joule, conduccion termica, conveccion, radiacion y limites termicos del material.
Modelo matematico	?Que ecuaciones representan el fenomeno?	Ecuacion de calor transitoria con fuente volumetrica dependiente de corriente electrica.
Modelo computacional	?Como se discretiza y resuelve el modelo?	Diferencias finitas, elementos finitos o volumen finito.
Simulacion	?Que escenarios se evaluan?	Corrientes, tiempos de exposicion, materiales y condiciones ambientales.
Prototipo	?Que implementacion material o digital permite verificar la idea?	Ensayo fisico, banco de prueba, software reproducible o prototipo experimental.
Validacion	?Que tan bien coincide el modelo con evidencia independiente?	Comparacion con mediciones de temperatura, dano observable o curvas experimentales.


Figura 1: Cadena metodologica entre fenomeno real, modelo, simulacion, prototipo y validacion.

1.4. Una primera simulacion: estimacion de π por Monte Carlo

Antes de entrar en modelos basados en ecuaciones diferenciales, conviene introducir un ejemplo minimo de simulacion estocastica. El metodo Monte Carlo se basa en sustituir un calculo determinista por un experimento numerico repetido sobre variables aleatorias. Su utilidad excede al ejemplo didactico: aparece en propagacion de incertidumbre, integracion en alta dimension, optimizacion estocastica, fisica de particulas, finanzas computacionales, trazado de rayos y validacion de modelos.

1.4.1. Planteamiento

Sea el cuadrado unitario $\Omega_S = [0, 1] \times [0, 1]$ y el circulo inscrito Ω_C de centro $(\frac{1}{2}, \frac{1}{2})$ y radio $r = \frac{1}{2}$. Por construccion,

$$\frac{|\Omega_C|}{|\Omega_S|} = \frac{\pi r^2}{(2r)^2} = \frac{\pi}{4}. \quad (6)$$

Si se generan N pares $\{(x_i, y_i)\}_{i=1}^N$ con $x_i, y_i \sim \mathcal{U}(0, 1)$ independientes, y N_{in} es el numero de muestras que satisfacen $(x_i - \frac{1}{2})^2 + (y_i - \frac{1}{2})^2 \leq r^2$, entonces el estimador

$$\hat{\pi}_N = 4 \frac{N_{\text{in}}}{N} \quad (7)$$

es un estimador insesgado de π . La ley fuerte de los grandes numeros garantiza $\hat{\pi}_N \rightarrow \pi$ casi seguramente cuando $N \rightarrow \infty$.

1.4.2. Tasa de convergencia y limitaciones

La variable aleatoria $\mathbb{K}_{\Omega_C}(x, y)$ es Bernoulli con parametro $p = \pi/4$. Por tanto la varianza del estimador es

$$\text{Var}[\hat{\pi}_N] = 16 \frac{p(1-p)}{N} \approx \frac{16 \cdot (\pi/4)(1 - \pi/4)}{N} \approx \frac{2.697}{N}, \quad (8)$$

y la desviacion estandar decae como $\sigma \propto N^{-1/2}$. Esta tasa es independiente de la dimension del problema (a diferencia de las cuadraturas de Newton-Cotes, cuyo error crece exponencialmente con la dimension). Por eso Monte Carlo es competitivo en integrales de alta dimension, pero ineficiente para ganar dos cifras decimales adicionales (cada cifra extra requiere $\sim 100\times$ mas muestras).

1.4.3.Codigo de ejemplo (Google Colab)

```

2      Estimacion del numero PI mediante simulacion Monte Carlo
3      .
4      Geometria: cuadrado [0,1]x[0,1] con circulo inscrito de
5      radio r=0.5
6      centrado en (0.5, 0.5).
7
8      Principio: si se generan N puntos uniformes en el
9      cuadrado y N_in
10     caen dentro del circulo, entonces:
11     Area_circulo / Area_cuadrado = pi*r^2 / (2r)^2 = pi/4
12     => pi ~ 4 * N_in / N
13
14     Convergencia teorica: error ~ 1/sqrt(N) (Ley de los
15     grandes numeros).
16     """
17
18     import numpy as np
19     import matplotlib.pyplot as plt
20
21     # ---- Parametros de la simulacion
22     -----
23     np.random.seed(42)
24     N = 100_000                # numero total de muestras
25     r = 0.5                   # radio del circulo
26     inscrito
27     cx, cy = 0.5, 0.5        # centro del circulo
28
29     # ---- Generacion de puntos uniformes en el cuadrado
30     -----
31     x = np.random.uniform(0.0, 1.0, N)
32     y = np.random.uniform(0.0, 1.0, N)
33
34     # ---- Indicador de pertenencia al circulo
35     -----
36     dentro = (x - cx)**2 + (y - cy)**2 <= r**2
37     N_in    = np.count_nonzero(dentro)
38
39     # ---- Estimador de pi
40     -----
41     pi_est  = 4.0 * N_in / N
42     err_abs = abs(pi_est - np.pi)
43     err_rel = err_abs / np.pi

```

```

35
36     print(f"N          = {N}")
37     print(f"N_in       = {N_in}")
38     print(f"pi_est      = {pi_est:.6f}")
39     print(f"pi_real     = {np.pi:.6f}")
40     print(f"err_abs     = {err_abs:.2e}")
41     print(f"err_rel     = {err_rel:.2e}")
42
43     # ---- Convergencia: pi estimado vs numero de muestras
44     -----
45     n_grid = np.unique(np.logspace(1, np.log10(N), 80).
46     astype(int))
47     pi_seq = np.array([4.0 * np.count_nonzero(dentro[:n]) /
48     n for n in n_grid])
49
50     # ---- Visualizacion
51     -----
52     fig, axes = plt.subplots(1, 2, figsize=(12, 5))
53
54     # Panel 1: nube de puntos
55     ax = axes[0]
56     ax.scatter(x[dentro][:5000], y[dentro][:5000], s=2, c=
57     'tab:blue',
58     label=f'dentro ({N_in}/{N})')
59     ax.scatter(x[~dentro][:5000], y[~dentro][:5000], s=2, c=
60     'tab:red',
61     label=f'fuera ({N-N_in}/{N})')
62     theta = np.linspace(0, 2*np.pi, 400)
63     ax.plot(cx + r*np.cos(theta), cy + r*np.sin(theta), 'k-'
64     , lw=1.5)
65     ax.set_aspect('equal'); ax.set_xlim(0, 1); ax.set_ylim
66     (0, 1)
67     ax.set_xlabel('x'); ax.set_ylabel('y')
68     ax.set_title(f'Monte Carlo en el cuadrado unitario\
69     napi_est = {pi_est:.5f}')
70     ax.legend(loc='upper right', fontsize=8); ax.grid(alpha
71     =0.3)
72
73     # Panel 2: convergencia
74     ax = axes[1]
75     ax.semilogx(n_grid, pi_seq, '-', lw=1.2, label=r'$\pi_{
76     est}(N)$')

```

```

66     ax.axhline(np.pi, color='k', ls='--', lw=1, label=r'$\pi$')
67     ax.fill_between(n_grid, np.pi - 4/np.sqrt(n_grid),
68                    np.pi + 4/np.sqrt(n_grid),
69                    alpha=0.15, color='gray', label=r'$\pm 4/\sqrt{N}$')
70     ax.set_xlabel('N'); ax.set_ylabel(r'$\pi_{est}$')
71     ax.set_title('Convergencia del estimador Monte Carlo')
72     ax.grid(True, which='both', alpha=0.3); ax.legend()
73
74     plt.tight_layout()
75     plt.show()
76

```

Listing 1: Estimacion de π por Monte Carlo con analisis de convergencia.

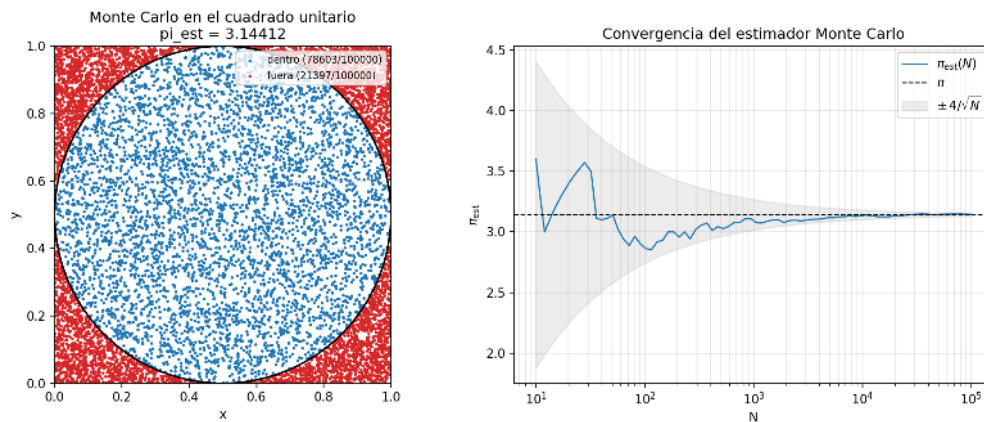


Figura 2: Estimacion de π por Monte Carlo con $N = 10^5$ muestras. Panel izquierdo: nube de puntos dentro (azul) y fuera (rojo) del circulo inscrito; el cociente $4N_{in}/N$ aproxima π . Panel derecho: convergencia del estimador con la banda teorica $\pm 4/\sqrt{N}$ que acota tipicamente la dispersion. El resultado con $N = 10^5$ es $\hat{\pi} \approx 3.144$, error relativo $\sim 10^{-3}$.

1.4.4. Lectura metodologica

Aunque elemental, este ejemplo ilustra cinco puntos transferibles al resto del curso:

1. **Semilla aleatoria.** Sin `np.random.seed` el resultado cambia cada corrida, lo cual destruye reproducibilidad. La semilla no *mejora* la estimacion; solo la hace verificable.
2. **Tasa de convergencia.** El error decae como $N^{-1/2}$. Es un techo teorico; cualquier mejora requiere otra estrategia (quasi-Monte Carlo, importance sampling, control variates).

3. **Costo computacional.** Para reducir el error relativo de 10^{-3} a 10^{-5} se necesita $10^4 \times$ mas muestras. Conviene declararlo antes de prometer precision.
4. **Verificacion vs validacion.** Se conoce π con precision arbitraria, por lo que este es un caso de *verificacion* (el codigo resuelve correctamente las matematicas), no de *validacion* (las matematicas describen correctamente la realidad). Ambas son etapas distintas y obligatorias.
5. **Generalizacion.** Sustituyendo el indicador del circulo por un integrando arbitrario $f(\mathbf{x})$ y normalizando por el volumen del dominio, se obtiene la formula general de integracion Monte Carlo en $\mathbb{R}^d$.

1.5. Diferencia entre modelo fisico, matematico y computacional

1.5.1. Modelo fisico

Un modelo fisico es una representacion conceptual del fenomeno basada en mecanismos causales. No necesariamente contiene ecuaciones completas, pero si define que mecanismos se consideran relevantes. En un problema termico, el modelo fisico puede establecer que la temperatura cambia por conduccion interna, generacion de calor y perdidas hacia el ambiente. En un problema estructural, puede establecer que el desplazamiento depende de rigidez, carga, geometria y restricciones.

Un modelo fisico responde preguntas del tipo:

- ¿Que variables interactuan?
- ¿Que mecanismos se consideran dominantes?
- ¿Que procesos se omiten y por que?
- ¿En que intervalo espacial y temporal se considera valido el analisis?

Por ejemplo, para una bomba centrifuga de sangre, el modelo fisico puede declarar que el flujo se considera incompresible, viscoso y gobernado por gradientes de presion y fuerzas inerciales. A partir de esa declaracion se justifica el uso de las ecuaciones de Navier–Stokes. La simplificacion fisica no es un defecto; es una operacion necesaria. El defecto aparece cuando la simplificacion no se declara, porque entonces el modelo se convierte en una caja negra con pretensiones de ciencia.

1.6. Modelo matematico

Un modelo matematico formaliza el modelo fisico mediante ecuaciones, dominios, restricciones, condiciones iniciales y condiciones de frontera. De forma general, un modelo gobernado por ecuaciones diferenciales parciales puede escribirse como

$$\mathcal{R}(\mathbf{u}, \frac{\partial \mathbf{u}}{\partial t}, \nabla \mathbf{u}, \nabla^2 \mathbf{u}; \boldsymbol{\theta}) = 0, \quad \mathbf{x} \in \Omega, \quad t \in (0, T], \quad (9)$$

$$\mathcal{B}(\mathbf{u}; \boldsymbol{\theta}) = \mathbf{g}, \quad \mathbf{x} \in \partial\Omega, \quad (10)$$

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}), \quad \mathbf{x} \in \Omega. \quad (11)$$

Aqui, $\mathcal{R}$ representa el residuo del modelo, $\mathcal{B}$ representa el operador de frontera, $\partial\Omega$ es la frontera del dominio, $\mathbf{g}$ son valores prescritos o flujos impuestos, y $\mathbf{u}_0$ es la condicion inicial.

1.6.1. Formulacion fuerte vs formulacion debil

La formulacion anterior se denomina *fuerte*: exige que la ecuacion se cumpla puntualmente y que la solucion sea suficientemente diferenciable. Para problemas con co-

eficientes discontinuos, geometrias complejas, o fuentes puntuales, esta regularidad es excesiva. La formulacion *debil* relaja el requisito: se busca $\mathbf{u}$ en un espacio de Sobolev $H^1(\Omega)$ tal que, para toda funcion de prueba $\mathbf{v} \in V$ con $\mathbf{v} = \mathbf{0}$ en la frontera de Dirichlet,

$$\int_{\Omega} \mathcal{R}(\mathbf{u}) \cdot \mathbf{v} \, d\Omega = 0. \quad (12)$$

Esta formulacion es la base del Metodo de Elementos Finitos (FEM), del metodo de Galerkin y de buena parte de la teoria de redes neuronales informadas por fisica. La existencia y unicidad en formulacion debil se prueba habitualmente mediante el teorema de Lax–Milgram cuando la forma bilineal asociada es coerciva y continua.

1.6.2. Adimensionalizacion como diagnostico

Antes de discretizar, conviene escribir el modelo en forma adimensional. Esto no es esteticismo: la adimensionalizacion revela los grupos dominantes (Reynolds, Peclet, Fourier, Biot, Damkohler, etc.) y permite identificar regimenes asintoticos donde el modelo se puede simplificar (parametros pequenos $\rightarrow$ teoria de perturbaciones, parametros grandes $\rightarrow$ aproximaciones de capa limite). Numericamente, ademas, mejora el condicionamiento de las matrices porque los terminos quedan de orden $O(1)$.

1.7. Modelo computacional

Un modelo computacional es la implementacion discreta del modelo matematico. Implica elegir una malla, un paso temporal, un esquema de integracion, una estructura de datos, un algoritmo de solucion y criterios de convergencia. Por ejemplo, una ecuacion continua como

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2} \quad (13)$$

no puede ejecutarse directamente en una computadora. Debe transformarse en una expresion discreta. Si se usa un esquema explicito de diferencias finitas, se obtiene

$$T_i^{n+1} = T_i^n + r (T_{i+1}^n - 2T_i^n + T_{i-1}^n), \quad r = \frac{\alpha \Delta t}{\Delta x^2}. \quad (14)$$

El parametro r controla la estabilidad del metodo. Para el esquema explicito unidimensional, se requiere

$$r \leq \frac{1}{2}. \quad (15)$$

Este ejemplo muestra una idea clave: el modelo fisico puede ser correcto y el modelo matematico puede estar bien planteado, pero una mala discretizacion puede producir resultados numericamente falsos. La computadora no se equivoca con mala intencion; solo obedece instrucciones absurdas con una lealtad preocupante.

1.7.1. Familias de esquemas y propiedades minimas

La eleccion del esquema temporal no es cuestion de preferencia estetica. Las familias principales para un problema $\dot{\mathbf{u}} = f(\mathbf{u}, t)$ son:

- **Explicitos** (Euler explicito, Runge–Kutta 4): baratos por paso, condicionalmente estables. Aceptables para problemas no-rigidos con dinamica suave.
- **Implicitos** (Euler implicito, Crank–Nicolson, BDF): incondicionalmente estables (al menos en sentido A -estable para algunos miembros), pero requieren resolver un sistema (no lineal en general) en cada paso. Obligatorios en problemas rigidos.
- **Simplecticos** (Verlet, leap-frog): preservan estructura hamiltoniana en sistemas mecanicos conservativos. Imprescindibles si se simulan tiempos largos sin amortiguamiento.
- **Adaptativos** (Runge–Kutta–Fehlberg, Dormand–Prince): ajustan Δt a una tolerancia local. Standard de hecho en `scipy.integrate.solve_ivp`.

Tres propiedades deben verificarse en todo esquema antes de aceptar sus resultados:

1. **Consistencia.** El error local de truncamiento tiende a cero cuando $\Delta t, \Delta x \rightarrow 0$.
2. **Estabilidad.** Pequeñas perturbaciones no se amplifican sin control (análisis de von Neumann, criterio CFL).
3. **Convergencia.** Por el teorema de Lax (problemas lineales bien planteados), *consistencia + estabilidad $\Rightarrow$ convergencia*.

1.8. Abstraccion y simplificacion de fenomenos reales

La abstraccion consiste en decidir que elementos del fenomeno se preservan y cuales se eliminan. No se modela la realidad completa; se modela una estructura interpretativa de la realidad. Esta decision debe estar guiada por el objetivo de investigacion.

Sea $\mathcal{S}$ el sistema real completo. Un modelo se puede entender como una proyeccion

$$\Pi : \mathcal{S} \rightarrow \mathcal{M}, \quad (16)$$

donde Π elimina informacion considerada secundaria y preserva informacion considerada relevante. El error de modelado aparece porque Π nunca es una identidad. Formalmente, si $\mathbf{u}$ es el estado real y $\hat{\mathbf{u}}$ es el estado predicho, el error total puede expresarse como

$$\mathbf{e}(\mathbf{x}, t) = \mathbf{u}(\mathbf{x}, t) - \hat{\mathbf{u}}(\mathbf{x}, t). \quad (17)$$

El problema doctoral consiste en justificar por que ese error es aceptable para la decision cientifica o tecnologica que se desea tomar.

1.8.1. Teorema II de Buckingham y reduccion sistematica

La simplificacion no es arbitraria. El teorema II de Buckingham establece que si una relacion fisica involucra n variables dimensionales y k dimensiones fundamentales independientes (por ejemplo M, L, T, Θ), entonces puede expresarse en terminos de $n - k$ grupos adimensionales $\pi_1, \dots, \pi_{n-k}$. Este resultado convierte la pregunta “¿cuales variables importan?” en “¿cuales grupos adimensionales son grandes, pequenos o de orden uno?”. Los grupos con valores extremos sugieren simplificaciones; los grupos de orden uno representan competencia entre mecanismos y obligan a conservarlos.

1.8.2. Analisis de sensibilidad

Una vez declarado el modelo $\hat{\mathbf{u}} = \mathcal{M}(\boldsymbol{\theta}, \mathbf{c})$, conviene cuantificar cuanto influye cada parametro en la salida. Se distinguen tres niveles:

- **Sensibilidad local** (one-at-a-time): derivada parcial $\partial \hat{\mathbf{u}} / \partial \theta_i$. Util pero limitada porque ignora interacciones.
- **Sensibilidad global de varianza** (indices de Sobol): descompone la varianza total de la salida en contribuciones individuales y de interaccion,

$$S_i = \frac{\text{Var}_{\theta_i} [\mathbb{E}_{\boldsymbol{\theta}_{\sim i}} [\hat{\mathbf{u}} \mid \theta_i]]}{\text{Var}[\hat{\mathbf{u}}]}, \quad (18)$$

con $\sum_i S_i \leq 1$ (igualdad si no hay interacciones).

- **Sensibilidad por screening** (Morris): identifica con pocas evaluaciones cuales parámetros tienen efecto despreciable, para descartarlos antes de un análisis costoso.

Realizar al menos un screening preliminar es buena practica antes de invertir en una calibracion bayesiana o en un metamodelo: descartar parametros irrelevantes simplifica el modelo y mejora la identifiabilidad.

1.8.3. Ejemplo de abstraccion: cable OPGW sometido a cortocircuito

Un cable OPGW real tiene geometria helicoidal, multiples materiales, contacto entre alambres, intercambio termico con el aire, variabilidad ambiental, propiedades dependientes de la temperatura y posibles danos acumulativos. Un primer modelo termico simplificado puede reducir el problema a una barra equivalente con temperatura promedio $T(x, t)$. Si se considera conduccion axial, generacion termica por efecto Joule y conveccion con el ambiente, se obtiene

$$\rho c_p A \frac{\partial T}{\partial t} = k A \frac{\partial^2 T}{\partial x^2} + I^2 R' - h P (T - T_\infty), \quad (19)$$

donde ρ es la densidad, c_p es el calor especifico, A es el area transversal equivalente, k es la conductividad termica, I es la corriente, R' es la resistencia electrica por unidad de longitud, h es el coeficiente convectivo, P es el perimetro expuesto y T_∞ es la temperatura ambiente.

Esta ecuacion no representa todos los detalles del cable, pero puede responder una pregunta concreta: ¿cuanto aumenta la temperatura promedio bajo una corriente de cortocircuito durante un tiempo determinado? Si el objetivo fuera estudiar dano local en alambres individuales, el modelo seria insuficiente. Por lo tanto, la validez del modelo depende del objetivo.

1.8.4. Ejemplo de abstraccion: sistema RAG como prototipo computacional

En un sistema de generacion aumentada por recuperacion, el fenomeno no es fisico en sentido clasico, sino informacional. El estado de entrada se representa mediante una consulta q , una base documental $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$ y una funcion de similitud $s(q, d_i)$. La recuperacion puede modelarse como

$$\mathcal{D}_k(q) = \text{TopK}_{d_i \in \mathcal{D}} s(q, d_i), \quad (20)$$

y la respuesta generada como

$$\hat{a} = G(q, \mathcal{D}_k(q); \theta), \quad (21)$$

donde G es el generador y θ son sus parametros. Aunque no hay una ecuacion diferencial, si existe un modelo computacional formal: entrada, recuperacion, representacion, generacion, metrica y validacion. Este tipo de ejemplo es util para evitar la falsa idea de que simulacion solo significa resolver calor, fluidos o estructuras.

1.9. Variables, parametros y condiciones de frontera

1.9.1. Variables de estado

Las variables de estado describen la condicion interna del sistema. Se denotan mediante u y pueden depender del espacio, el tiempo o ambos. Ejemplos:

- Temperatura: $T(\mathbf{x}, t)$.
- Desplazamiento mecanico: $\mathbf{u}(\mathbf{x}, t)$.
- Velocidad de fluido: $\mathbf{v}(\mathbf{x}, t)$.
- Concentracion quimica: $C(\mathbf{x}, t)$.
- Estado de agente: $s_i(t)$ en un modelo basado en agentes.
- Distribucion de probabilidad de texto: $p(w_t \mid w_{<t}, q, \mathcal{D}_k)$ en un modelo generativo.

Una mala seleccion de variables de estado produce modelos debiles. Por ejemplo, si se modela una bomba de sangre unicamente con caudal y presion, pero se omiten esfuerzos cortantes relacionados con dano celular, el modelo puede ser hidraulicamente aceptable y biomedicamente insuficiente. Esta distincion es crucial en investigacion doctoral, porque la variable que se mide no siempre es la variable que importa.

1.9.2. Parametros

Los parametros son cantidades que caracterizan el sistema pero no evolucionan como variables principales dentro de la simulacion, aunque pueden depender de temperatura, frecuencia, escala o condiciones de operacion. Se agrupan como

$$\boldsymbol{\theta} = \{\theta_1, \theta_2, \dots, \theta_p\}. \quad (22)$$

Ejemplos: conductividad termica k , modulo de Young E , densidad ρ , viscosidad dinamica μ , coeficiente convectivo h , paso de malla Δx , tamano de lote en entrenamiento, numero de agentes, probabilidad de transicion o umbral de decision.

Debe distinguirse entre parametros fisicos y parametros numericos. Un parametro fisico describe el sistema; un parametro numerico describe la solucion computacional. Confundirlos lleva a interpretaciones pobres. Por ejemplo, k representa conductividad termica; Δx representa resolucion espacial. Cambiar k cambia el fenomeno modelado; cambiar Δx cambia la aproximacion computacional.

Un nivel adicional util es distinguir entre *parametros nominales* (valores de catalogo o de referencia bibliografica), *parametros calibrados* (estimados a partir de datos del propio experimento) y *parametros operativos* (controles que el usuario fija). Solo los segundos son objeto legitimo de un problema inverso; los primeros tienen incertidumbre acotada por la fuente; los terceros son inputs del escenario.

1.9.3. Condiciones iniciales y de frontera

Las condiciones iniciales especifican el estado del sistema en $t = 0$:

$$\mathbf{u}(\mathbf{x}, 0) = \mathbf{u}_0(\mathbf{x}). \quad (23)$$

Las condiciones de frontera especifican como interactua el dominio con el exterior. En problemas de campo, las condiciones mas frecuentes son:

Tabla 2: Tipos comunes de condiciones de frontera.

Tipo	Forma matematica	Interpretacion
Dirichlet	$u = u_D$ en $\partial\Omega$	Se prescribe el valor de la variable. Ejemplo: temperatura fija.
Neumann	$\nabla u \cdot \mathbf{n} = q_N$ en $\partial\Omega$	Se prescribe el flujo normal. Ejemplo: flujo de calor impuesto.
Robin	$au + b\nabla u \cdot \mathbf{n} = g$	Se prescribe una combinacion de valor y flujo. Ejemplo: conveccion termica.
Periodica	$u(0, t) = u(L, t)$	El dominio se conecta consigo mismo. Ejemplo: celda repetitiva.
Simetria	$\nabla u \cdot \mathbf{n} = 0$	Caso particular de Neumann homogenea para explotar simetria geometrica.
Absorbente (ABC, PML)	operador que evita reflexion	Frontera artificial truncada en problemas hiperbolicos o de propagacion no acotados.

Para un problema termico con conveccion, la condicion de Robin puede expresarse como

$$-k\nabla T \cdot \mathbf{n} = h(T - T_\infty), \quad \mathbf{x} \in \partial\Omega. \quad (24)$$

Esta ecuacion indica que el flujo termico saliente es proporcional a la diferencia entre la temperatura superficial y la temperatura ambiente. No es un adorno matematico: define como el modelo se comunica con el exterior.

Una observacion practica: la mala especificacion de las condiciones de frontera es probablemente la primera causa de simulaciones que “casi funcionan”. Pueden producir resultados visualmente plausibles pero cuantitativamente incorrectos. Conviene siempre verificar el flujo neto en $\partial\Omega$ y compararlo con un balance global; si no cierra, las condiciones de frontera estan mal definidas o mal implementadas.

1.10. Escalas espaciales, temporales y numeros adimensionales

Un modelo científico debe declarar su escala de validez. Una ecuación que describa un fenómeno a escala macroscópica puede fallar a escala microscópica. Un modelo estacionario puede ser razonable en procesos lentos y absurdo en procesos transitorios rápidos. Por ello se emplean escalas características.

Sea L_c una longitud característica, T_c un tiempo característico, U_c una velocidad característica y Θ_c una escala de temperatura. Se definen variables adimensionales como

$$\mathbf{x}^* = \frac{\mathbf{x}}{L_c}, \quad t^* = \frac{t}{T_c}, \quad \mathbf{v}^* = \frac{\mathbf{v}}{U_c}, \quad T^* = \frac{T - T_\infty}{\Theta_c}. \quad (25)$$

La adimensionalización permite identificar grupos dominantes. Por ejemplo, en transferencia de calor transitoria aparece el numero de Fourier:

$$Fo = \frac{\alpha t}{L_c^2}, \quad (26)$$

donde $\alpha = k/(\rho c_p)$ es la difusividad térmica. Si $Fo \ll 1$, el proceso térmico apenas ha penetrado el dominio; si $Fo \gg 1$, el sistema puede aproximarse al régimen cercano al equilibrio térmico.

En dinámica de fluidos aparece el numero de Reynolds:

$$Re = \frac{\rho U_c L_c}{\mu}, \quad (27)$$

donde ρ es la densidad, μ la viscosidad dinámica y U_c una velocidad característica. Este número relaciona fuerzas inerciales con fuerzas viscosas. En una bomba centrífuga, Re ayuda a anticipar si el flujo será laminar, transicional o turbulento, aunque por sí solo no resuelve el problema geométrico ni la interacción rotor–fluido.

En transferencia de calor con convección aparece el numero de Biot:

$$Bi = \frac{h L_c}{k}. \quad (28)$$

Si $Bi \ll 0.1$, la temperatura interna tiende a ser relativamente uniforme y puede usarse un modelo lumped; si Bi no es pequeño, se requiere resolver gradientes internos. Este criterio evita usar modelos de parámetros concentrados cuando el fenómeno exige un modelo distribuido.

1.10.1. Otros grupos adimensionales relevantes

La siguiente tabla resume otros números adimensionales que aparecen con frecuencia en ingeniería y que conviene tener presentes al formular un modelo:

Tabla 3: Numeros adimensionales relevantes en ingenieria.

Numero	Definicion	Interpretacion
Prandtl	$Pr = \mu c_p / k$	Razon entre difusividad de momento y difusividad termica. Es propiedad del fluido.
Peclet	$Pe = Re Pr = U_c L_c / \alpha$	Adveccion frente a difusion termica. $Pe \gg 1$: transporte por adveccion domina.
Nusselt	$Nu = h L_c / k_f$	Conveccion frente a conduccion en el fluido. Salida tipica de correlaciones.
Mach	$M = U / c$	Velocidad de flujo frente a velocidad del sonido. $M < 0.3$ permite tratar el flujo como incompresible.
Strouhal	$St = f L_c / U_c$	Frecuencia caracteristica de desprendimiento de vortices o forzamiento.
Damkohler	$Da = \tau_{\text{flujo}} / \tau_{\text{reacc}}$	Tiempo de residencia frente a tiempo de reaccion en flujos reactivos.
Knudsen	$Kn = \lambda_{mfp} / L_c$	Camino libre medio frente a escala macroscopica. $Kn > 0.01$ invalida la hipotesis continua.
Weber	$We = \rho U_c^2 L_c / \sigma$	Inercia frente a tension superficial. Relevante en interfases.

Una recomendacion practica: antes de simular, estimar los grupos relevantes con valores tipicos del problema. Si un numero esta lejos de la unidad por dos o tres ordenes de magnitud, hay una simplificacion natural disponible; si esta cerca de la unidad, hay competencia entre mecanismos y el modelo *tiene* que conservar ambos terminos.

1.10.2. Escala numerica y condicion CFL

En simulacion transitoria, la relacion entre paso espacial y paso temporal controla estabilidad y precision. Para muchos problemas hiperbolicos y parabolicos aparece una condicion tipo Courant–Friedrichs–Lewy (CFL). Para una ecuacion de transporte unidimensional,

$$\frac{\partial u}{\partial t} + c \frac{\partial u}{\partial x} = 0, \quad (29)$$

un esquema explicito requiere tipicamente

$$C = \frac{c \Delta t}{\Delta x} \leq C_{\text{máx}}. \quad (30)$$

El numero C indica cuantas celdas avanza la informacion fisica durante un paso temporal. Si el algoritmo permite que la informacion salte mas alla de lo que la malla puede representar, la simulacion se descompone. La inestabilidad numerica no es una propiedad del fenomeno; es una propiedad del metodo usado para aproximarlos.

Para la ecuación de difusión en 2D, la condición equivalente es $r_x + r_y \leq 1/2$, con $r_i = \alpha \Delta t / \Delta x_i^2$. Este criterio se invoca en el Ejemplo 7 del intercambiador de calor y debe verificarse explícitamente en el código.

1.11. Error, precision e incertidumbre

1.11.1. Error total

En simulacion cientifica, el error total no proviene de una sola fuente. Puede descomponerse conceptualmente como

$$e_{\text{total}} = e_{\text{modelo}} + e_{\text{parametros}} + e_{\text{discretizacion}} + e_{\text{iterativo}} + e_{\text{redondeo}} + e_{\text{medicion}}. \quad (31)$$

Cada termino tiene naturaleza diferente:

- e_{modelo} : proviene de hipotesis fisicas incompletas o incorrectas.
- $e_{\text{parametros}}$: proviene de valores inciertos de propiedades o constantes.
- $e_{\text{discretizacion}}$: proviene de representar un dominio continuo mediante una malla finita.
- $e_{\text{iterativo}}$: proviene de detener un algoritmo antes de convergencia exacta.
- e_{redondeo} : proviene de precision finita de la computadora.
- e_{medicion} : proviene de ruido, sesgo o limitaciones instrumentales.

La precision numerica no garantiza exactitud fisica. Una simulacion puede tener muchas cifras decimales y ser fisicamente inutil. Ese vicio aparece con frecuencia cuando se confunde estetica computacional con evidencia cientifica.

1.11.2. Orden de convergencia y extrapolacion de Richardson

Para un esquema consistente de orden p , el error de discretizacion sobre una variable de interes ϕ se comporta como

$$\phi_h = \phi_{\text{exacto}} + C h^p + O(h^{p+1}), \quad (32)$$

donde h es un parametro caracteristico de la malla (tamano de elemento, paso temporal). Realizando tres corridas con tamanos $h_1 < h_2 < h_3$ y factor de refinamiento $r = h_2/h_1 = h_3/h_2$, el orden observado se estima como

$$p_{\text{obs}} = \frac{\ln [(\phi_3 - \phi_2)/(\phi_2 - \phi_1)]}{\ln r}, \quad (33)$$

y la extrapolacion de Richardson permite estimar el valor refinado,

$$\phi_{\text{exacto}} \approx \phi_1 + \frac{\phi_1 - \phi_2}{r^{p_{\text{obs}}} - 1}. \quad (34)$$

El *Grid Convergence Index* (GCI), propuesto por Roache y adoptado en ASME V&V 20, proporciona una banda de incertidumbre asociada al error de discretización:

$$GCI = F_s \frac{|\phi_1 - \phi_2|}{(r^{p_{\text{obs}}} - 1) |\phi_1|}, \quad (35)$$

con $F_s \approx 1.25$ (factor de seguridad). Reportar GCI junto al resultado convierte una simulación en un dato cuantificable; sin el, es solo un gráfico bonito.

1.11.3. Verificación y validación (V&V)

La comunidad de simulación distingue formalmente dos procesos:

- **Verificación.** ¿Resuelve el código correctamente las ecuaciones que se le pidió resolver? Se compara contra soluciones manufacturadas, soluciones analíticas, o convergencia de malla. Es un problema matemático.
- **Validación.** ¿Resuelve el código las ecuaciones correctas para el fenómeno real? Se compara contra mediciones experimentales con incertidumbre conocida. Es un problema físico.

Una simulación verificada pero no validada puede ser internamente consistente y físicamente incorrecta. Una simulación validada pero no verificada puede acertar por compensación de errores. Ambas etapas son obligatorias antes de usar un modelo para decisiones de ingeniería. El estándar ASME V&V 10/20 y la guía NASA-STD-7009 formalizan este flujo.

1.11.4. Métricas de comparación

Si y_i son datos experimentales y $\hat{y}_i$ son predicciones del modelo, se definen métricas como el error absoluto medio:

$$MAE = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad (36)$$

el error cuadrático medio:

$$MSE = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2, \quad (37)$$

y la raíz del error cuadrático medio:

$$RMSE = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}. \quad (38)$$

Tambien puede emplearse una norma relativa:

$$e_{rel} = \frac{\|\mathbf{y} - \hat{\mathbf{y}}\|_2}{\|\mathbf{y}\|_2}. \quad (39)$$

Estas metricas no son equivalentes. El RMSE penaliza errores grandes; el MAE es mas robusto ante valores extremos. En validacion doctoral, debe justificarse la metrica elegida de acuerdo con el dano, costo o consecuencia de los errores.

Para datos espaciales o temporales correlacionados conviene complementar con el coeficiente de determinacion R^2 , el indice de Willmott d o, en imagenes, SSIM. Para distribuciones de probabilidad: divergencia de Kullback–Leibler o distancia de Wasserstein. La eleccion de la metrica condiciona el resultado: una norma cuadratica favorece ajustes globales; una norma infinito penaliza fallas locales severas.

1.11.5. Propagacion de incertidumbre

Si una magnitud de salida z depende de variables de entrada $\mathbf{x} = [x_1, x_2, \dots, x_n]^T$, es decir,

$$z = f(\mathbf{x}), \quad (40)$$

y las entradas tienen matriz de covarianza Σ_x , la aproximacion lineal de primer orden permite estimar

$$\sigma_z^2 \approx \mathbf{J} \Sigma_x \mathbf{J}^T, \quad (41)$$

donde

$$\mathbf{J} = \left[\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial x_2}, \dots, \frac{\partial f}{\partial x_n} \right] \quad (42)$$

es el Jacobiano de sensibilidad. Para una salida vectorial $\mathbf{z} = \mathbf{f}(\mathbf{x})$, se obtiene

$$\Sigma_z \approx \mathbf{J} \Sigma_x \mathbf{J}^T. \quad (43)$$

Esta expresion es central para conectar metrologia, simulacion y validacion experimental. Permite cuantificar como errores de entrada se transforman en incertidumbre de salida.

La linealizacion es valida solo si f es localmente diferenciable y las incertidumbres son pequenas. Cuando hay no-linealidad fuerte, distribuciones no gaussianas o discontinuidades, se debe recurrir a propagacion de Monte Carlo (analogia al ejemplo de la Seccion 2): se generan muestras de $\mathbf{x}$ segun su distribucion conjunta, se evalua f para cada muestra, y se construyen estadisticas empiricas de z . Esta tecnica enlaza directamente el primer ejemplo del documento con la cuantificacion de incertidumbre en problemas reales.

Las principales fuentes de error presentes en simulacion computacional se resumen en la [Figura 3](#).

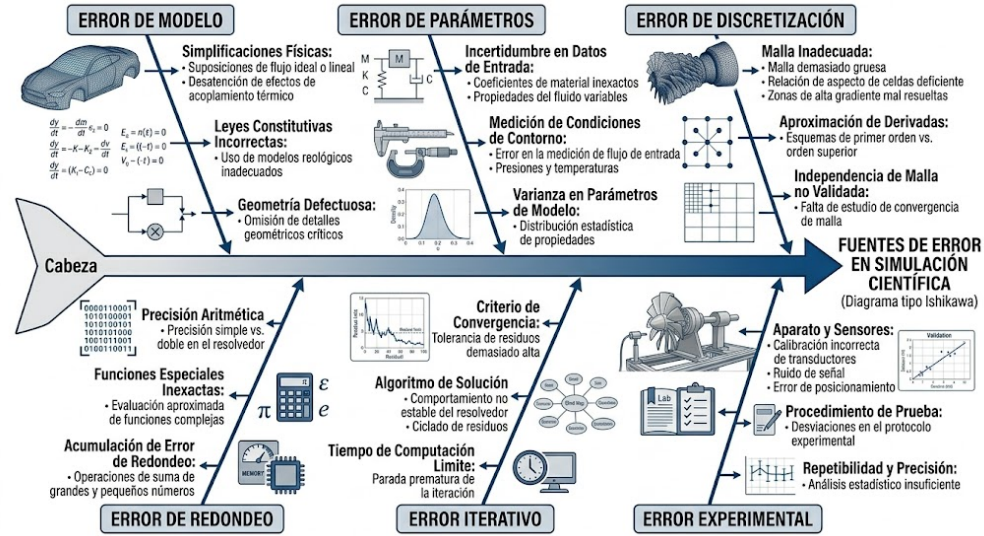


Figura 3: Principales fuentes de error en simulación científica y modelado computacional.

1.12. Reproducibilidad científica

Un resultado reproducible es aquel que puede obtenerse nuevamente bajo condiciones claramente documentadas. En simulación y prototipado, la reproducibilidad requiere más que compartir una gráfica. Deben reportarse:

- versión del código;
- lenguaje y bibliotecas empleadas;
- unidades de todas las variables;
- parámetros físicos y numéricos;
- condiciones iniciales y de frontera;
- criterios de convergencia;
- estructura de datos;
- semilla aleatoria, si aplica;
- archivos de entrada;
- instrucciones mínimas para ejecutar.

Un resultado sin unidades, sin parámetros, sin malla y sin código verificable se parece más a una opinión con gráfica que a una simulación científica. Para evitarlo, se recomienda que cada práctica del curso produzca al menos cuatro elementos: archivo de código, figura exportada, tabla de parámetros y breve interpretación técnica.

Las prácticas FAIR (Findable, Accessible, Interoperable, Reusable) para datos científicos se trasladan naturalmente al código: repositorio público con DOI (Zenodo, Figshare), licencia explícita, archivo `environment.yml` o `requirements.txt` con versiones fijas, contenedores reproducibles (Docker, Apptainer) cuando hay dependencias complejas, y notebooks ejecutables (Google Colab, Binder) para la difusión docente.

1.12.1. Registro mínimo de una simulación

Toda simulación entregada durante el curso deberá incluir una tabla como la siguiente:

Tabla 4: Registro minimo para reproducibilidad de una simulacion.

Elemento	Descripcion requerida
Fenomeno modelado	Descripcion fisica, biologica, social o computacional del sistema.
Objetivo de simulacion	Pregunta especifica que responde la simulacion.
Variables de estado	Variables dependientes y unidades.
Parametros	Valores, unidades y fuente o justificacion.
Hipotesis	Simplificaciones explicitas y limitaciones.
Dominio	Geometria, intervalo temporal y escala.
Condiciones iniciales/-frontera	Valores prescritos, flujos o restricciones.
Metodo numerico	Diferencias finitas, FEM, Monte Carlo, agentes, redes neuronales, etc.
Resolucion	Tamano de malla, paso temporal o numero de muestras.
Criterio de validacion	Metrica, dato de referencia o prueba de consistencia.
Codigo	Lenguaje, bibliotecas y semilla aleatoria.
Resultado	Graficas, tablas y conclusion tecnica.

La validacion cuantitativa entre simulacion y experimento se representa conceptualmente en la **Figura 4**.

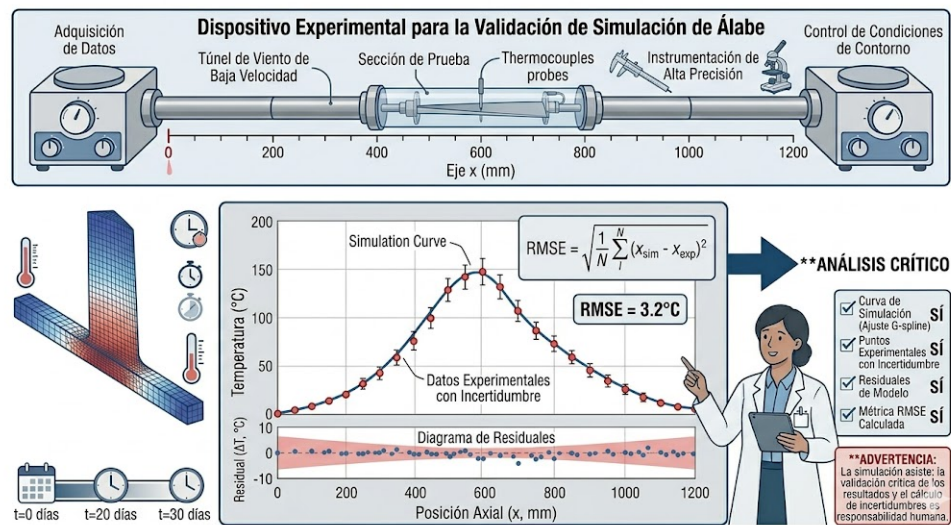


Figura 4: Comparacion conceptual entre resultados experimentales y simulacion computacional con analisis de residuos.

1.13. Ejemplo 1: conduccion transitoria en una barra

1.13.1. Planteamiento fisico

Se analiza una barra de longitud L sometida a una distribucion inicial de temperatura. Los extremos se mantienen a temperatura ambiente T_0 . Se desea simular la evolucion temporal de la temperatura y observar la disipacion del gradiente termico.

1.13.2. Modelo matematico

Se emplea la ecuacion de calor unidimensional:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}, \quad 0 < x < L, \quad t > 0. \quad (44)$$

Las condiciones de frontera son de Dirichlet:

$$T(0, t) = T_0, \quad T(L, t) = T_0. \quad (45)$$

La condicion inicial se define como una perturbacion gaussiana:

$$T(x, 0) = T_0 + A \exp \left[-\frac{(x - x_c)^2}{2\sigma^2} \right], \quad (46)$$

donde A es la amplitud termica, x_c es el centro de la perturbacion y σ controla su ancho.

1.13.3. Modelo computacional

Se discretiza el dominio con N nodos:

$$x_i = i\Delta x, \quad \Delta x = \frac{L}{N-1}, \quad i = 0, 1, \dots, N-1. \quad (47)$$

El tiempo se discretiza como

$$t^n = n\Delta t. \quad (48)$$

El esquema explicito queda:

$$T_i^{n+1} = T_i^n + r (T_{i+1}^n - 2T_i^n + T_{i-1}^n), \quad r = \frac{\alpha \Delta t}{\Delta x^2}. \quad (49)$$

Para estabilidad se impone $r \leq 0.5$.

1.13.4. Codigo base en Python

```
import numpy as np
```

```
2      import matplotlib.pyplot as plt
3
4      # Parametros fisicos
5      L = 1.0                # longitud [m]
6      alpha = 1.0e-4         # difusividad termica [m^2/s]
7      T0 = 25.0              # temperatura ambiente [degC]
8      A = 80.0               # amplitud de perturbacion [degC]
9      xc = 0.5               # centro de perturbacion [m]
10     sigma = 0.06           # ancho de perturbacion [m]
11
12     # Parametros numericos
13     N = 151
14     x = np.linspace(0, L, N)
15     dx = x[1] - x[0]
16     r = 0.45
17     dt = r * dx**2 / alpha
18     nt = 800
19
20     # Condicion inicial
21     T = T0 + A*np.exp(-((x-xc)**2)/(2*sigma**2))
22     T[0] = T0
23     T[-1] = T0
24
25     snapshots = []
26     times = []
27     for n in range(nt):
28         Tnew = T.copy()
29         Tnew[1:-1] = T[1:-1] + r*(T[2:] - 2*T[1:-1] + T[:-2])
30         Tnew[0] = T0
31         Tnew[-1] = T0
32         T = Tnew
33         if n in [0, 50, 150, 300, 799]:
34             snapshots.append(T.copy())
35             times.append(n*dt)
36
37     plt.figure(figsize=(7, 4))
38     for Tplot, t in zip(snapshots, times):
39         plt.plot(x, Tplot, label=f't = {t:.2f} s')
40         plt.xlabel('x [m]')
41         plt.ylabel('Temperatura [degC]')
42         plt.title('Conduccion transitoria 1D')
43         plt.grid(True)
```

```

44 plt.legend()
45 plt.tight_layout()
46 plt.show()
47

```

Listing 2: Simulacion explicita de conduccion transitoria 1D.

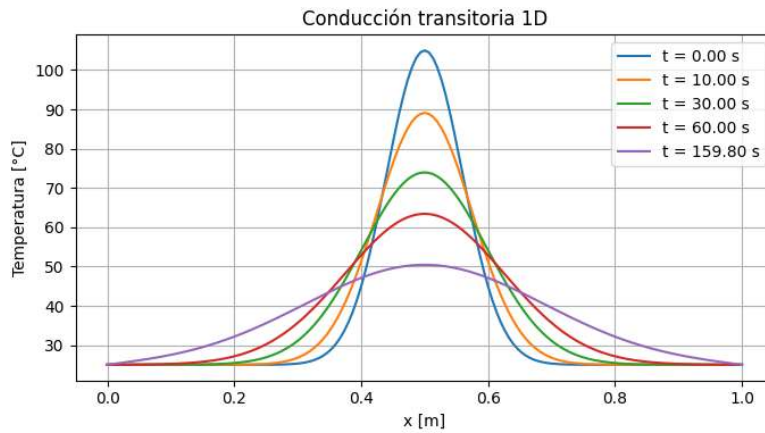


Figura 5: Resultado de la Simulacion explicita de conduccion transitoria 1D.

La evolucion temporal de la distribucion termica obtenida mediante diferencias finitas se ilustra conceptualmente en la [Figura 6](#).

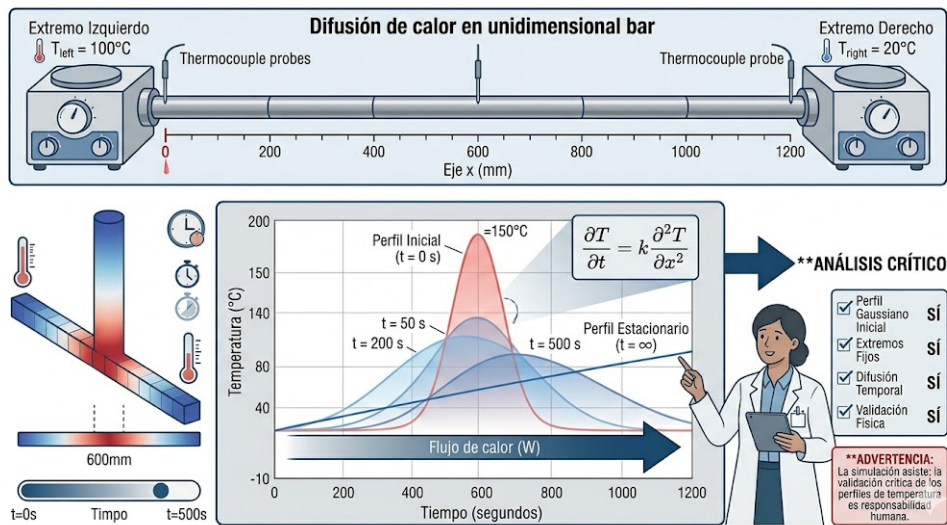


Figura 6: Difusion temporal de un perfil termico en una barra unidimensional.

1.13.5. Interpretacion esperada

La perturbacion termica se suaviza con el tiempo. La temperatura maxima disminuye y el perfil se ensancha debido a la difusion. Si se aumenta α , la disipacion ocurre mas rapido. Si se viola $r \leq 0.5$, el perfil puede oscilar o divergir, lo cual evidencia inestabilidad numerica. El fenomeno fisico no exploto; exploto el metodo.

1.14. Ejemplo 2: sistema masa–resorte–amortiguador

1.14.1. Planteamiento fisico

Se considera un sistema de un grado de libertad compuesto por una masa m , un resorte de rigidez k y un amortiguador viscoso con coeficiente c . El sistema se somete a una fuerza externa $F(t)$. Este ejemplo permite conectar modelo fisico, ecuacion diferencial ordinaria, integracion numerica y analisis parametrico.

1.14.2. Modelo matematico

La segunda ley de Newton conduce a

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = F(t), \quad (50)$$

donde $x(t)$ es el desplazamiento. Se definen la frecuencia natural no amortiguada y la razon de amortiguamiento:

$$\omega_n = \sqrt{\frac{k}{m}}, \quad \zeta = \frac{c}{2\sqrt{km}}. \quad (51)$$

Si $F(t) = 0$, el comportamiento libre depende de ζ . Para $0 < \zeta < 1$, el sistema es subamortiguado y presenta oscilacion decreciente.

Para resolver numericamente, se convierte la ecuacion de segundo orden en un sistema de primer orden:

$$\begin{cases} \dot{x} = v, \\ \dot{v} = \frac{1}{m} [F(t) - cv - kx]. \end{cases} \quad (52)$$

1.14.3. Discretizacion mediante Euler explicito

Un esquema elemental se escribe como

$$x^{n+1} = x^n + \Delta t v^n, \quad (53)$$

$$v^{n+1} = v^n + \Delta t \frac{F^n - cv^n - kx^n}{m}. \quad (54)$$

Este metodo es simple, pero puede ser inestable si Δt es demasiado grande. La eleccion de metodo numerico debe formar parte del razonamiento cientifico, no de la decoracion del codigo.

1.14.4. Actividad breve de analisis

Se pide comparar tres valores de amortiguamiento: $\zeta = 0.05$, $\zeta = 0.2$ y $\zeta = 0.8$. Para cada caso se debe graficar $x(t)$ y discutir como cambia la disipacion de energia. La energia mecanica total se calcula como

$$E(t) = \frac{1}{2}mv(t)^2 + \frac{1}{2}kx(t)^2. \quad (55)$$

En un sistema amortiguado, $E(t)$ debe decrecer. Si la energia crece sin fuerza externa, el algoritmo esta introduciendo energia numerica artificial.

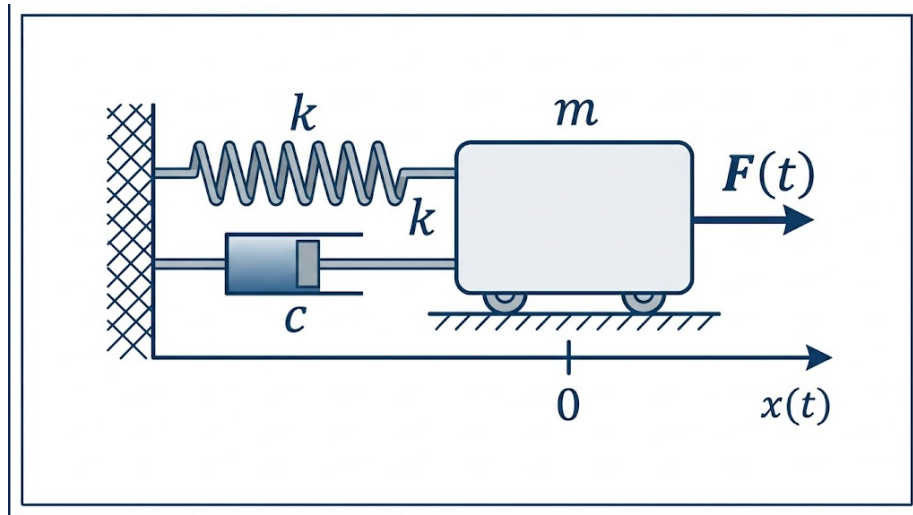


Figura 7: Representacion esquematica de un sistema masa–resorte–amortiguador horizontal.

1.15. Ejemplo 3: modelo termico simplificado para cable OPGW

1.15.1. Motivacion

Los articulos sobre cables OPGW permiten discutir una transicion natural entre investigacion experimental y simulacion numerica. El cable real presenta una arquitectura compleja, pero en una primera aproximacion puede formularse un modelo termico de parametros concentrados si se supone temperatura uniforme en la seccion transversal.

1.15.2. Modelo concentrado

Si se ignoran gradientes espaciales internos, el balance energetico se expresa como

$$mc_p \frac{dT}{dt} = I^2 R - hA_s(T - T_\infty) - \varepsilon \sigma_{SB} A_s (T^4 - T_\infty^4), \quad (56)$$

donde m es la masa del tramo considerado, c_p el calor especifico, I la corriente, R la resistencia electrica, h el coeficiente convectivo, A_s el area superficial, ε la emisividad y σ_{SB} la constante de Stefan–Boltzmann.

Si la radiacion se desprecia para temperaturas moderadas, se obtiene

$$mc_p \frac{dT}{dt} = I^2 R - hA_s(T - T_\infty). \quad (57)$$

Esta ecuacion lineal tiene solucion analitica:

$$T(t) = T_\infty + \frac{I^2 R}{hA_s} + \left[T(0) - T_\infty - \frac{I^2 R}{hA_s} \right] \exp \left(-\frac{hA_s}{mc_p} t \right). \quad (58)$$

1.15.3. Criterio de validez

El modelo concentrado exige que los gradientes internos sean pequenos. Un criterio clasico es

$$Bi = \frac{hL_c}{k} \ll 0.1. \quad (59)$$

Si Bi no es pequeno, debe utilizarse un modelo distribuido, por ejemplo mediante elementos finitos o diferencias finitas. Esta decision ilustra el papel del modelado: no se elige el metodo mas sofisticado por vanidad computacional, sino el que responde la pregunta con el nivel de fidelidad requerido.

1.15.4. Prompt para que el estudiante genere código con IA

Prompt recomendado. Genera un script en Python para simular el calentamiento transitorio de un cable OPGW mediante un modelo termico concentrado. Usa el balance $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Define parametros con unidades, integra la ecuacion con un metodo numerico explicito, grafica $T(t)$ para tres valores de corriente I , calcula la temperatura maxima y verifica si se supera un umbral critico definido por el usuario. Incluye comentarios tecnicos en espanol y una tabla impresa con los parametros usados.

1.16. Ejemplo 4: bomba centrífuga y dinamica de fluidos computacional

1.16.1. Modelo fisico

Una bomba centrífuga transforma energia mecanica rotacional en energia hidraulica. En aplicaciones biomedicas, ademas del caudal y la presion, se debe considerar el dano potencial a la sangre debido a esfuerzos cortantes. Este ejemplo permite introducir modelos de campo vectorial y criterios de desempeno.

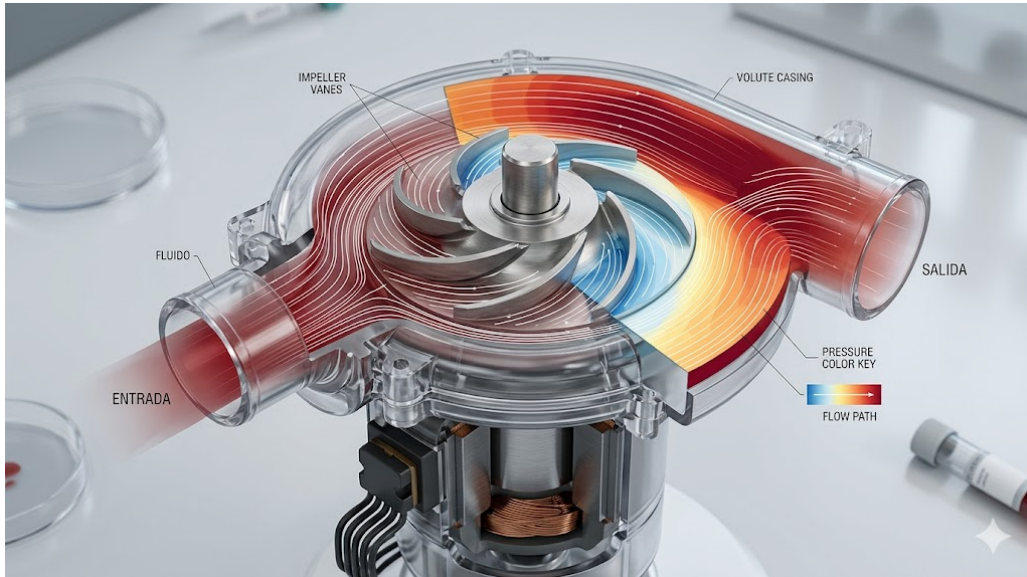


Figura 8: Representación conceptual de una bomba centrífuga biomedica analizada mediante dinamica de fluidos computacional.

1.16.2. Ecuaciones gobernantes

Para un fluido newtoniano, incompresible e isotermico, se emplean las ecuaciones de Navier–Stokes:

$$\nabla \cdot \mathbf{v} = 0, \quad (60)$$

$$\rho \left(\frac{\partial \mathbf{v}}{\partial t} + \mathbf{v} \cdot \nabla \mathbf{v} \right) = -\nabla p + \mu \nabla^2 \mathbf{v} + \mathbf{f}, \quad (61)$$

donde $\mathbf{v}$ es la velocidad, p la presión, ρ la densidad, μ la viscosidad dinamica y $\mathbf{f}$ una fuerza volumetrica.

1.16.3. Indicadores de desempeño

La carga hidráulica $H = \Delta p / (\rho g)$, la eficiencia $\eta = \rho g Q H / P_{in}$ y un indicador de exposición a esfuerzo cortante $D = \int_0^{t_e} \tau(t)^a dt$ permiten comparar diseños antes de prototipar.

1.16.4. Prototipado

geometria CAD $\rightarrow$ malla $\rightarrow$ condiciones de frontera $\rightarrow$ solución CFD $\rightarrow$ curvas $Q-H$ $\rightarrow$ validación.
(62)

1.17. Ejemplo 5: modelo basado en agentes para economía circular

1.17.1. Planteamiento

No todos los sistemas de interés doctoral son continuos ni gobernados por ecuaciones diferenciales. En economía circular, adopción tecnológica, logística, movilidad o comportamiento organizacional, puede ser más adecuado representar agentes con reglas de decisión.

Sea un conjunto de agentes $\mathcal{A} = \{a_1, a_2, \dots, a_N\}$. Cada agente tiene un estado $s_i(t)$, un conjunto de atributos θ_i y una regla de decisión

$$s_i(t+1) = \Phi_i(s_i(t), \mathcal{N}_i(t), \theta_i, \xi_i(t)), \quad (63)$$

donde $\mathcal{N}_i(t)$ representa vecinos o interacciones y $\xi_i(t)$ representa aleatoriedad.

1.17.2. Ejemplo: adopción de una tecnología de reciclaje

Cada agente puede estar en uno de dos estados: no adopta (0) o adopta (1) una tecnología. La probabilidad de adopción puede definirse como

$$P[s_i(t+1) = 1] = \sigma(\beta_0 + \beta_1 B_i - \beta_2 C_i + \beta_3 A_{\mathcal{N}_i}), \quad (64)$$

con $\sigma(z) = 1/(1 + e^{-z})$.

1.17.3. Código conceptual mínimo

```

1      import numpy as np
2      import matplotlib.pyplot as plt
3
4      np.random.seed(7)
5      N = 200
6      steps = 80
7
8      benefit = np.random.normal(1.0, 0.2, N)
9      cost = np.random.normal(1.0, 0.25, N)
10     state = np.zeros(N, dtype=int)
11     state[np.random.choice(N, 5, replace=False)] = 1
12
13     beta0, beta1, beta2, beta3 = -1.2, 1.5, 1.2, 2.0
14
15     adoption_fraction = []
16     for t in range(steps):
17         adoption_fraction.append(state.mean())
    
```

```
18     new_state = state.copy()
19     global_adoption = state.mean()
20     for i in range(N):
21         z = beta0 + beta1*benefit[i] - beta2*cost[i] + beta3*
global_adoption
22         p = 1/(1 + np.exp(-z))
23         if np.random.rand() < p:
24             new_state[i] = 1
25             state = new_state
26
27     plt.figure(figsize=(7,4))
28     plt.plot(adoption_fraction)
29     plt.xlabel('Paso temporal'); plt.ylabel('Fraccion de
adopcion')
30     plt.title('Modelo basado en agentes: adopcion
tecnologica')
31     plt.grid(True); plt.tight_layout(); plt.show()
32
```

Listing 3: Modelo simplificado de adopcion tecnologica mediante agentes.

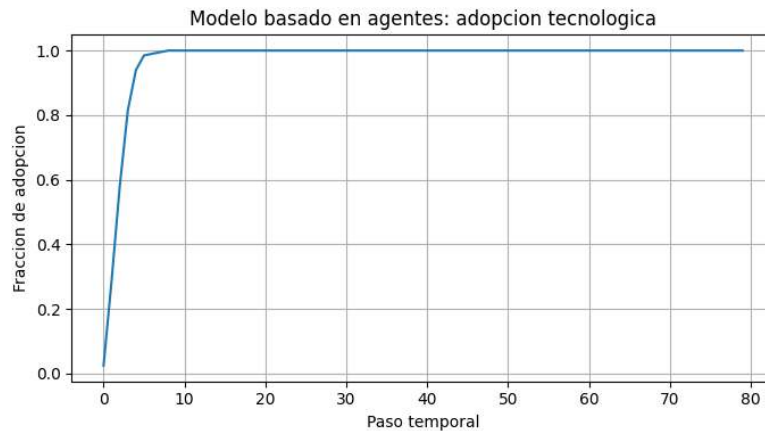


Figura 9: Resultado Modelo simplificado de adopcion tecnologica mediante agentes.

1.18. Ejemplo 6: red neuronal informada por fisica

1.18.1. Idea central

Una red neuronal informada por fisica (PINN) no solo ajusta datos; minimiza simultaneamente error de datos y violacion de una ecuacion gobernante. Para una ecuacion diferencial parcial general,

$$\mathcal{R} \left(u, \frac{\partial u}{\partial t}, \nabla u, \nabla^2 u; \boldsymbol{\theta} \right) = 0, \quad (65)$$

se propone una red neuronal $u_\theta(\mathbf{x}, t)$ y se construye una funcion de perdida:

$$\mathcal{L} = \lambda_d \mathcal{L}_{data} + \lambda_r \mathcal{L}_{res} + \lambda_b \mathcal{L}_{bc} + \lambda_i \mathcal{L}_{ic}, \quad (66)$$

donde

$$\mathcal{L}_{data} = \frac{1}{N_d} \sum_{j=1}^{N_d} |u_\theta(\mathbf{x}_j, t_j) - u_j^{obs}|^2, \quad (67)$$

$$\mathcal{L}_{res} = \frac{1}{N_r} \sum_{j=1}^{N_r} |\mathcal{R}(u_\theta(\mathbf{x}_j, t_j))|^2, \quad (68)$$

$$\mathcal{L}_{bc} = \frac{1}{N_b} \sum_{j=1}^{N_b} |\mathcal{B}(u_\theta(\mathbf{x}_j, t_j)) - g_j|^2, \quad (69)$$

$$\mathcal{L}_{ic} = \frac{1}{N_i} \sum_{j=1}^{N_i} |u_\theta(\mathbf{x}_j, 0) - u_0(\mathbf{x}_j)|^2. \quad (70)$$

Este enfoque exige definir fenomeno, ecuacion, datos, dominio, condiciones y validacion. Una PINN mal formulada es solo una red neuronal cara intentando adivinar fisica con cara de paper.

1.18.2. Ejemplo con ecuacion de calor

Para la ecuacion

$$\frac{\partial u}{\partial t} - \alpha \frac{\partial^2 u}{\partial x^2} = 0, \quad (71)$$

el residuo fisico es $r_\theta(x, t) = \frac{\partial u_\theta}{\partial t} - \alpha \frac{\partial^2 u_\theta}{\partial x^2}$. La calidad del modelo se evalua comparando u_θ con una solucion analitica, datos experimentales o una simulacion numerica de referencia.

1.19. Ejemplo 7: conduccion 2D en placa de intercambiador de calor

1.19.1. Motivacion fisica

Los intercambiadores de calor de placas son uno de los componentes mas comunes en ingenieria termica. Cada placa metalica delgada se encuentra en contacto con dos fluidos a temperaturas diferentes y conduce calor lateralmente a lo largo de su area. El analisis transitorio de una placa permite responder preguntas como: ¿cuanto tarda en alcanzarse el regimen permanente?, ¿donde aparecen los maximos gradientes (zonas criticas de fatiga termica)?, ¿como cambia la distribucion si se modifican los caminos de fluido?

Este ejemplo amplia el caso 1D del Ejemplo 1 a una placa rectangular 2D y muestra de manera explicita la transicion entre el estado inicial uniforme y el estado estable gobernado por la ecuacion de Laplace.

1.19.2. Modelo matematico

Sea $\Omega = [0, L_x] \times [0, L_y]$ el dominio rectangular de la placa, y $T(x, y, t)$ la temperatura. Sin generacion volumetrica de calor, la ecuacion gobernante es la ecuacion de calor 2D:

$$\frac{\partial T}{\partial t} = \alpha \left(\frac{\partial^2 T}{\partial x^2} + \frac{\partial^2 T}{\partial y^2} \right), \quad \alpha = \frac{k}{\rho c_p}. \quad (72)$$

Condiciones de frontera de tipo Dirichlet, que representan paredes en contacto termico con los fluidos:

$$T(0, y, t) = T_h, \quad T(L_x, y, t) = T_c, \quad T(x, 0, t) = T_c, \quad T(x, L_y, t) = T_h. \quad (73)$$

Condicion inicial uniforme:

$$T(x, y, 0) = T_0. \quad (74)$$

Cuando $t \rightarrow \infty$ la derivada temporal se anula y el campo $T_\infty(x, y)$ resuelve la ecuacion de Laplace $\nabla^2 T = 0$ con las condiciones de frontera anteriores; este es el estado estable que se busca alcanzar.

1.19.3. Discretizacion explicita FTCS 2D

Sobre una malla estructurada de $N_x \times N_y$ nodos con pasos $\Delta x, \Delta y$, el esquema de diferencias finitas centrado en espacio y explicito en tiempo (FTCS) es

$$T_{i,j}^{n+1} = T_{i,j}^n + r_x (T_{i+1,j}^n - 2T_{i,j}^n + T_{i-1,j}^n) + r_y (T_{i,j+1}^n - 2T_{i,j}^n + T_{i,j-1}^n), \quad (75)$$

con $r_x = \alpha \Delta t / \Delta x^2$, $r_y = \alpha \Delta t / \Delta y^2$. El análisis de estabilidad de von Neumann impone

$$r_x + r_y \leq \frac{1}{2}. \quad (76)$$

Para $\Delta x = \Delta y$, esto equivale a $\alpha \Delta t / \Delta x^2 \leq 1/4$, mas restrictivo que en 1D.

1.19.4. Criterio de estado estable

Numericamente, se considera alcanzado el regimen permanente cuando

$$\max_{i,j} |T_{i,j}^{n+1} - T_{i,j}^n| < \text{tol}, \quad (77)$$

con tol típicamente entre 10^{-4} y 10^{-6} veces la escala de temperatura del problema.

1.19.5. Código en Python (Google Colab)

```

1      """
2      Conduccion transitoria 2D en la placa de un
      intercambiador de calor.
3
4      Ecuacion gobernante:
5      dT/dt = alpha * (d2T/dx2 + d2T/dy2)
6
7      Dominio:      Omega = [0,Lx] x [0,Ly]
8      Material:      placa metalica (acero estructural por
      defecto)
9      Condiciones de frontera (Dirichlet, lado caliente / lado
      frio):
10     T(0, y, t)      = T_hot           (entrada de fluido caliente)
11     T(Lx, y, t)     = T_cold          (salida lado frio)
12     T(x, 0, t)      = T_cold          (pared lateral fria)
13     T(x, Ly, t)     = T_hot           (pared lateral caliente)
14     Condicion inicial:
15     T(x, y, 0)      = T0              (placa uniformemente a
      temperatura ambiente)
16
17     Discretizacion: diferencias finitas explicitas (FTCS) en
      una malla
18     estructurada Nx x Ny.
19
20     Criterio de estabilidad (von Neumann) en 2D:
21     r_x + r_y <= 1/2,      r_i = alpha * dt / dx_i^2.
    
```

```

22
23     Captura 5 estados:
24     snapshot 1: t = 0                (estado inicial)
25     snapshot 5: estado estable (||Tn+1 - Tn||_inf < tol
26 )
27
28     snapshots 2-4: instantes intermedios distribuidos
29     logarítmicamente
30     entre t=0 y t_ss
31     """
32
33     import numpy as np
34     import matplotlib.pyplot as plt
35     from matplotlib import cm
36
37     # ===== Parametros fisicos
38     =====
39
40     Lx, Ly    = 0.20, 0.10            # dimensiones placa [m]
41     (20 cm x 10 cm)
42     k_th      = 50.0                 # conductividad termica
43     [W/(m K)] (acero)
44     rho       = 7850.0               # densidad
45     [kg/m3]
46     cp        = 480.0                # calor especifico
47     [J/(kg K)]
48     alpha     = k_th / (rho * cp)    # difusividad termica
49     [m2/s]
50
51     T_hot     = 200.0                # temperatura lado
52     caliente [degC]
53     T_cold    = 20.0                 # temperatura lado frio
54     [degC]
55     T0        = 25.0                 # temperatura inicial
56     placa [degC]
57
58     # ===== Parametros numericos
59     =====
60
61     Nx, Ny    = 161, 81              # nodos en x e y
62     dx        = Lx / (Nx - 1)
63     dy        = Ly / (Ny - 1)
64
65     # CFL: estabilidad explicita 2D
66     r_target  = 0.24                  # < 0.25 por seguridad

```

```

52     dt      = r_target * min(dx, dy)**2 / alpha
53     rx      = alpha * dt / dx**2
54     ry      = alpha * dt / dy**2
55     assert rx + ry <= 0.5, "Esquema inestable: rx + ry > 1/2
    "

56
57     print(f"alpha = {alpha:.3e} m^2/s")
58     print(f"dx = {dx*1e3:.3f} mm,  dy = {dy*1e3:.3f} mm,  dt
    = {dt:.4f} s")
59     print(f"rx = {rx:.4f},  ry = {ry:.4f},  rx+ry = {rx+ry
    :.4f}")

60
61     # ===== Inicializacion
    =====
62     T = np.full((Ny, Nx), T0, dtype=np.float64)
63
64     def aplicar_frontera(T):
65         T[:, 0]  = T_hot      # x = 0
66         T[:, -1] = T_cold     # x = Lx
67         T[0, :]  = T_cold     # y = 0
68         T[-1, :] = T_hot      # y = Ly
69         return T
70
71     T = aplicar_frontera(T)
72
73     # ===== Avance temporal hasta estado estable
    =====
74     tol_ss = 1e-4
75     n_max  = 200_000
76     T_hist = [T.copy()]
77     t_hist = [0.0]
78
79     t_now = 0.0
80     for n in range(1, n_max + 1):
81         Tn = T.copy()
82         T[1:-1, 1:-1] = (Tn[1:-1, 1:-1]
83 + rx*(Tn[1:-1, 2:] - 2*Tn[1:-1, 1:-1] + Tn[1:-1, :-2])
84 + ry*(Tn[2:, 1:-1] - 2*Tn[1:-1, 1:-1] + Tn[:-2, 1:-1]))
85         T = aplicar_frontera(T)
86         t_now += dt
87         if np.max(np.abs(T - Tn)) < tol_ss:
88             print(f"Estado estable: n = {n}, t = {t_now:.2f} s")

```

```

89         break
90
91         t_ss, T_ss = t_now, T.copy()
92
93         # ===== Re-ejecucion para capturar 3 instantes
intermedios ===
94         t_inter = np.geomspace(max(0.05*t_ss, 5*dt), 0.6*t_ss,
3)
95         T = aplicar_frontera(np.full((Ny, Nx), T0, dtype=np.
float64))
96         t_now, idx = 0.0, 0
97         snaps_mid, t_mid = [], []
98         while idx < len(t_inter):
99             Tn = T.copy()
100             T[1:-1, 1:-1] = (Tn[1:-1, 1:-1]
101 + rx*(Tn[1:-1, 2:] - 2*Tn[1:-1, 1:-1] + Tn[1:-1, :-2])
102 + ry*(Tn[2:, 1:-1] - 2*Tn[1:-1, 1:-1] + Tn[:-2, 1:-1]))
103             T = aplicar_frontera(T)
104             t_now += dt
105             if t_now >= t_inter[idx]:
106                 snaps_mid.append(T.copy()); t_mid.append(t_now); idx +=
1
107
108             T_hist += snaps_mid + [T_ss]
109             t_hist += t_mid + [t_ss]
110
111             # ===== Visualizacion
=====
112             x = np.linspace(0, Lx, Nx); y = np.linspace(0, Ly, Ny)
113             X, Y = np.meshgrid(x, y)
114
115             fig, axes = plt.subplots(5, 1, figsize=(8, 14),
constrained_layout=True)
116             for k, (Tk, tk, ax) in enumerate(zip(T_hist, t_hist,
axes)):
117                 cf = ax.pcolormesh(X*1e3, Y*1e3, Tk, shading='auto',
118 cmap=cm.inferno, vmin=T_cold, vmax=T_hot)
119                 ax.contour(X*1e3, Y*1e3, Tk, levels=10, colors='white',
120 linewidths=0.5, alpha=0.6)
121                 if k == 0:
122                     title = "t = 0.00 s (estado inicial)"
123                 elif k == 4:

```

```

124         title = f"t = {tk:.2f} s   (estado estable)"
125     else:
126         title = f"t = {tk:.2f} s   (transitorio)"
127     ax.set_title(title)
128     ax.set_xlabel('x [mm]'); ax.set_ylabel('y [mm]')
129     ax.set_aspect('equal')
130     fig.colorbar(cf, ax=axes, shrink=0.85, label='T [degC]')
131     fig.suptitle('Conduccion 2D en placa de intercambiador
de calor',
132                 fontsize=13, y=1.01)
133     plt.show()
134

```

Listing 4: Conduccion 2D en placa de intercambiador con cinco snapshots.

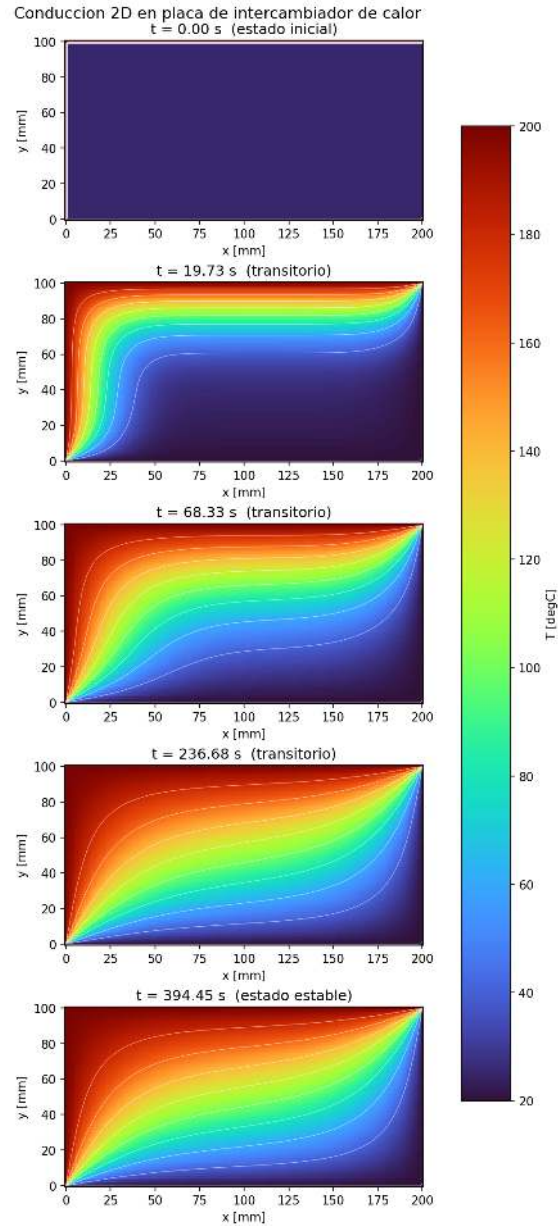


Figura 10: Conducción transitoria bidimensional en una placa de intercambiador de acero estructural con dimensiones $200 \text{ mm} \times 100 \text{ mm}$ y difusividad térmica $\alpha \approx 1.33 \times 10^{-5} \text{ m}^2/\text{s}$. Se presentan cinco estados térmicos: condición inicial uniforme a 25°C , tres estados transitorios distribuidos logarítmicamente y el estado estable alcanzado en $t \approx 394 \text{ s}$. El criterio de convergencia se define mediante una variación térmica máxima menor que 10^{-4}°C entre pasos consecutivos. Las isothermas en blanco muestran la transición desde gradientes térmicos intensos cerca de las fronteras calientes hasta una distribución suave gobernada por la ecuación de Laplace.

1.19.6. Lectura tecnica de los resultados

1. **Estado inicial** ($t = 0$). Campo uniforme a $T_0 = 25^\circ\text{C}$; las condiciones de frontera Dirichlet aun no se han propagado al interior. Los gradientes son singulares en

las esquinas, lo cual se manifiesta numericamente como esquinas brillantes en los primeros pasos.

2. **Transitorio temprano** ($t \sim 20$ s). El frente termico ha penetrado solo unos milímetros desde las fronteras calientes. La longitud de penetracion teorica es $\delta(t) \sim \sqrt{\alpha t} \approx 16$ mm, congruente con lo observado.
3. **Transitorios intermedios** ($t \sim 70\text{--}240$ s). Los frentes calientes desde $x = 0$ y desde $y = L_y$ se encuentran; aparece la estructura diagonal caracteristica de un problema con dos fronteras calientes en lados adyacentes.
4. **Estado estable** ($t \approx 394$ s). Las isothermas son curvas suaves; el campo ya no cambia mensurablemente. La temperatura media de la placa se estabiliza en ≈ 110 °C, intermedia entre T_h y T_c pero ponderada por la geometria.
5. **Tiempo caracteristico**. El numero de Fourier al estado estable es $Fo = \alpha t_{ss}/L_c^2 \approx 0.5$ con $L_c = \min(L_x, L_y)/2 = 50$ mm, consistente con la regla heuristica $Fo \sim 0.2\text{--}1$ para alcanzar el regimen permanente.

1.19.7. Limitaciones del modelo

El modelo asume conductividad y propiedades termicas constantes, espesor despreciable (problema estrictamente 2D), conveccion infinitamente eficiente en las fronteras (Dirichlet en lugar de Robin), ausencia de generacion volumetrica y ausencia de acoplamiento con el flujo de fluidos. Cualquiera de estos relajamientos genera un modelo distinto: conduccion no-lineal con $k(T)$, problema 3D, condiciones de Robin con h finito, problema termo-fluidodinamico acoplado, etc. La eleccion entre ellos depende de la pregunta de investigacion.

1.20. Ejemplo 8: analisis de esfuerzos de von Mises en una conexion viga-columna

1.20.1. Motivacion estructural

Las conexiones viga-columna concentran demanda mecanica: en un portico, la rigidez y la integridad del nudo determinan en gran medida la respuesta global. La esquina interior del nudo es una zona clasica de concentracion de esfuerzos, candidata al inicio de fluencia o fatiga. Este ejemplo ilustra como pasar de un modelo conceptual (“viga empujada en columna empujada”) a un modelo computacional 2D mediante el Metodo de Elementos Finitos con elementos triangulares de deformacion constante (CST, *Constant Strain Triangle*), y como obtener un mapa de esfuerzos de von Mises sobre la geometria deformada.

1.20.2. Modelo fisico y matematico

Se asume estado plano de esfuerzos (*plane stress*): la geometria es una placa delgada de espesor t en el plano xy , y los esfuerzos fuera del plano son despreciables. Esto es razonable para perfiles laminares delgados. Las ecuaciones gobernantes son la conservacion de momento en regimen estatico,

$$\nabla \cdot \boldsymbol{\sigma} + \mathbf{b} = \mathbf{0} \quad \text{en } \Omega, \quad (78)$$

junto con las relaciones constitutivas lineales elasticas isotropas

$$\boldsymbol{\sigma} = \mathbf{D} \boldsymbol{\varepsilon}, \quad \boldsymbol{\varepsilon} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T), \quad (79)$$

donde $\mathbf{u} = (u, v)$ es el desplazamiento. Para estado plano de esfuerzos la matriz constitutiva es

$$\mathbf{D} = \frac{E}{1 - \nu^2} \begin{bmatrix} 1 & \nu & 0 \\ \nu & 1 & 0 \\ 0 & 0 & \frac{1-\nu}{2} \end{bmatrix}. \quad (80)$$

Condiciones de frontera: empotramiento en la base de la columna ($u = v = 0$ en $y = 0$) y carga puntual descendente P en el extremo libre de la viga.

El esfuerzo equivalente de von Mises se define como

$$\sigma_{vm} = \sqrt{\sigma_x^2 - \sigma_x \sigma_y + \sigma_y^2 + 3\tau_{xy}^2}. \quad (81)$$

El criterio de fluencia de von Mises establece que el material entra en regimen plastico cuando $\sigma_{vm} \geq \sigma_y$ (limite elastico del material). Este criterio es el estandar de facto en aceros estructurales ductiles.

1.20.3. Discretización FEM con elementos CST

La formulación débil del problema conduce a la ecuación matricial $\mathbf{K}\mathbf{U} = \mathbf{F}$, donde $\mathbf{K}$ es la matriz de rigidez global y $\mathbf{F}$ el vector de cargas nodales. Para un elemento triangular CST de tres nodos con coordenadas (x_i, y_i) , $i = 1, 2, 3$, la matriz de deformación-desplazamiento es constante:

$$\mathbf{B} = \frac{1}{2A} \begin{bmatrix} b_1 & 0 & b_2 & 0 & b_3 & 0 \\ 0 & c_1 & 0 & c_2 & 0 & c_3 \\ c_1 & b_1 & c_2 & b_2 & c_3 & b_3 \end{bmatrix}, \quad (82)$$

con $b_i = y_j - y_k$, $c_i = x_k - x_j$ (índices cíclicos) y A el área del triángulo. La matriz de rigidez elemental es

$$\mathbf{K}^e = t A \mathbf{B}^T \mathbf{D} \mathbf{B} \in \mathbb{R}^{6 \times 6}. \quad (83)$$

Los esfuerzos se calculan como $\boldsymbol{\sigma}^e = \mathbf{D}\mathbf{B}\mathbf{u}^e$ y son constantes por elemento, lo cual exige promediado nodal ponderado por área para obtener una visualización suave (*nodal averaging*).

1.20.4. Limitaciones conocidas del CST

El elemento CST es el más simple posible en FEM 2D y resulta excelente para fines didácticos, pero tiene rigidez excesiva (*shear locking* parcial) y subestima desplazamientos cuando la malla es grosera. Para un análisis profesional se prefieren elementos de mayor orden (LST, Q4, Q8) o formulaciones mixtas. Aquí se acepta esta limitación a cambio de un código autocontenido sin dependencias externas pesadas.

1.20.5. Código en Python (Google Colab)

```

1
2     import numpy as np
3     import matplotlib.pyplot as plt
4     from matplotlib.tri import Triangulation
5     from scipy.sparse import lil_matrix, csr_matrix
6     from scipy.sparse.linalg import spsolve
7
8     # ===== Parametros del problema
9     =====
10    E      = 200.0e9          # modulo de Young [Pa]
11    nu     = 0.30            # Poisson
12    t      = 0.010          # espesor [m]
13    P      = 10.0e3          # carga vertical descendente [N]

```

```

13
14         wc      = 0.10                # ancho columna      [m]
15         Hc      = 0.80                # altura columna    [m]
16         Lb      = 0.80                # longitud viga     [m] desde
cara columna
17         hb      = 0.10                # peralte viga      [m]
18
19         elem_size = 0.02              # tamaño aproximado de
elemento [m]
20
21         # ===== Construcción de la geometría en L
=====
22         x_min, x_max = 0.0, wc + Lb
23         y_min, y_max = 0.0, Hc
24
25         nx = int(round((x_max - x_min) / elem_size)) + 1
26         ny = int(round((y_max - y_min) / elem_size)) + 1
27
28         xs = np.linspace(x_min, x_max, nx)
29         ys = np.linspace(y_min, y_max, ny)
30
31         Xg, Yg = np.meshgrid(xs, ys)
32
33         def in_domain(x, y):
34             in_col = (
35                 (x <= wc + 1e-12) &
36                 (y >= 0.0) &
37                 (y <= Hc + 1e-12)
38             )
39
40             in_beam = (
41                 (x >= wc - 1e-12) &
42                 (x <= wc + Lb + 1e-12) &
43                 (y >= Hc - hb - 1e-12) &
44                 (y <= Hc + 1e-12)
45             )
46
47             return in_col | in_beam
48
49         mask = in_domain(Xg, Yg)
50

```

```

51      # ===== Nodos
52      =====
53      node_idx = -np.ones_like(mask, dtype=int)
54      coords = []
55
56      for j in range(ny):
57      for i in range(nx):
58      if mask[j, i]:
59      node_idx[j, i] = len(coords)
60      coords.append((Xg[j, i], Yg[j, i]))
61
62      coords = np.array(coords)
63      n_nodes = coords.shape[0]
64
65      # ===== Elementos triangulares
66      =====
67      elements = []
68
69      for j in range(ny - 1):
70      for i in range(nx - 1):
71
72      n00 = node_idx[j, i]
73      n10 = node_idx[j, i+1]
74      n01 = node_idx[j+1, i]
75      n11 = node_idx[j+1, i+1]
76
77      if min(n00, n10, n01, n11) < 0:
78      continue
79
80      elements.append((n00, n10, n11))
81      elements.append((n00, n11, n01))
82
83      elements = np.array(elements, dtype=int)
84      n_elems = elements.shape[0]
85
86      print(f"Nodos: {n_nodes}, Elementos: {n_elems}")
87
88      # ===== Matriz constitutiva D para plane stress
89      =====
90
91      D = (E / (1.0 - nu**2)) * np.array([
92      [1.0, nu, 0.0],
93      [nu, 1.0, 0.0],
94      [0.0, 0.0, 0.5*(1-nu)]
95      ])

```

```

90         [0.0, 0.0, (1.0 - nu) / 2.0],
91     ])
92
93     # ===== Ensamblado de rigidez global
94     =====
95     K = lil_matrix((2*n_nodes, 2*n_nodes))
96
97     def stiffness_CST(xe, ye):
98         """Matriz de rigidez elemental para elemento triangular
99         CST."""
100
101         x1, x2, x3 = xe
102         y1, y2, y3 = ye
103
104         A2 = (x2 - x1)*(y3 - y1) - (x3 - x1)*(y2 - y1)
105         A = 0.5 * abs(A2)
106
107         b1 = y2 - y3
108         b2 = y3 - y1
109         b3 = y1 - y2
110
111         c1 = x3 - x2
112         c2 = x1 - x3
113         c3 = x2 - x1
114
115         B = (1.0 / A2) * np.array([
116             [b1, 0.0, b2, 0.0, b3, 0.0],
117             [0.0, c1, 0.0, c2, 0.0, c3],
118             [c1, b1, c2, b2, c3, b3],
119         ])
120
121         Ke = (B.T @ D @ B) * t * A
122
123         return Ke, B, A
124
125     elem_B = []
126     elem_area = []
127
128     for e, (n1, n2, n3) in enumerate(elements):
129
130         xe = coords[[n1, n2, n3], 0]
131         ye = coords[[n1, n2, n3], 1]

```

```

130
131     Ke, B, A = stiffness_CST(xe, ye)
132
133     elem_B.append(B)
134     elem_area.append(A)
135
136     dofs = [
137         2*n1, 2*n1 + 1,
138         2*n2, 2*n2 + 1,
139         2*n3, 2*n3 + 1
140     ]
141
142     for a in range(6):
143         for b in range(6):
144             K[dofs[a], dofs[b]] += Ke[a, b]
145
146     K = K.tocsr()
147
148     elem_B = np.array(elem_B)
149     elem_area = np.array(elem_area)
150
151     # ===== Condiciones de frontera
152     =====
153     fixed_nodes = np.where(np.isclose(coords[:, 1], 0.0))[0]
154     fixed_dofs = np.concatenate([2*fixed_nodes, 2*
155     fixed_nodes + 1])
156
157     # ===== Carga puntual
158     =====
159     x_load = wc + Lb
160     y_load = Hc - hb / 2.0
161
162     dist = (coords[:, 0] - x_load)**2 + (coords[:, 1] -
163     y_load)**2
164     load_node = int(np.argmin(dist))
165
166     print(
167         f"Nodo de carga: {load_node} en "
168         f"({coords[load_node, 0]:.3f}, {coords[load_node, 1]:.3f})
169     "
170     )

```

```

167     F = np.zeros(2*n_nodes)
168     F[2*load_node + 1] = -P
169
170     # ===== Resolución del sistema
171     =====
172     free_dofs = np.setdiff1d(np.arange(2*n_nodes),
173     fixed_dofs)
174
175     K_ff = K[free_dofs, :][:, free_dofs]
176     F_f = F[free_dofs]
177
178     u_f = spsolve(K_ff, F_f)
179
180     U = np.zeros(2*n_nodes)
181     U[free_dofs] = u_f
182
183     ux = U[0::2]
184     uy = U[1::2]
185
186     u_max = np.max(np.sqrt(ux**2 + uy**2))
187
188     print(f"Desplazamiento maximo: {u_max*1e3:.4f} mm")
189
190     # ===== Esfuerzos por elemento y von Mises
191     =====
192     sigma_elem = np.zeros((n_elems, 3))
193
194     for e, (n1, n2, n3) in enumerate(elements):
195
196         ue = np.array([
197             ux[n1], uy[n1],
198             ux[n2], uy[n2],
199             ux[n3], uy[n3]
200         ])
201
202         eps = elem_B[e] @ ue
203         sigma_elem[e] = D @ eps
204
205         sx, sy, txy = sigma_elem.T
206
207         vm_elem = np.sqrt(
208             sx**2 - sx*sy + sy**2 + 3.0*txy**2

```

```

206         )
207
208         # ===== Promediado nodal
209         =====
210         vm_node = np.zeros(n_nodes)
211         w_node  = np.zeros(n_nodes)
212
213         for e, (n1, n2, n3) in enumerate(elements):
214             for n in (n1, n2, n3):
215                 vm_node[n] += vm_elem[e] * elem_area[e]
216                 w_node[n]  += elem_area[e]
217
218         vm_node /= np.maximum(w_node, 1e-30)
219
220         print(f"von Mises maximo elemento: {vm_elem.max()/1e6
221               :.2f} MPa")
222         print(f"von Mises maximo nodal:      {vm_node.max()/1e6
223               :.2f} MPa")
224
225         # ===== Visualizacion: deformada amplificada
226         =====
227         tri = Triangulation(coords[:, 0], coords[:, 1], elements
228                             )
229
230         amp = 0.10 * max(Hc, wc + Lb) / max(u_max, 1e-12)
231
232         print(f"Factor amplificacion deformada: {amp:.1f}x")
233
234         coords_def = coords + amp * np.column_stack([ux, uy])
235
236         tri_def = Triangulation(
237             coords_def[:, 0],
238             coords_def[:, 1],
239             elements
240         )
241
242         fig, ax = plt.subplots(
243             figsize=(8, 6),
244             constrained_layout=True
245         )
246
247         tpc = ax.tripcolor(
    
```

```

243     tri_def,
244     vm_node / 1e6,
245     shading='gouraud',
246     cmap='turbo'
247 )
248
249     ax.triplot(
250     tri_def,
251     color='k',
252     lw=0.15,
253     alpha=0.3
254 )
255
256     ax.triplot(
257     tri,
258     color='gray',
259     lw=0.4,
260     alpha=0.4
261 )
262
263     ax.set_aspect('equal')
264     ax.set_xlabel('x [m]')
265     ax.set_ylabel('y [m]')
266
267     ax.set_title(
268     f'von Mises - deformada amplificada (x{amp:.0f})'
269 )
270
271     cbar = plt.colorbar(tpc, ax=ax)
272     cbar.set_label(r'$\sigma_{vm}$ [MPa]')
273
274     fig.suptitle(
275     f'Conexion viga-columna: estado plano de esfuerzos '
276     f'(CST, {n_elems} elementos)',
277     fontsize=13
278 )
279
280     plt.show()
281

```

Listing 5: Código Python para visualización de esfuerzos de von Mises en geometría deformada amplificada.

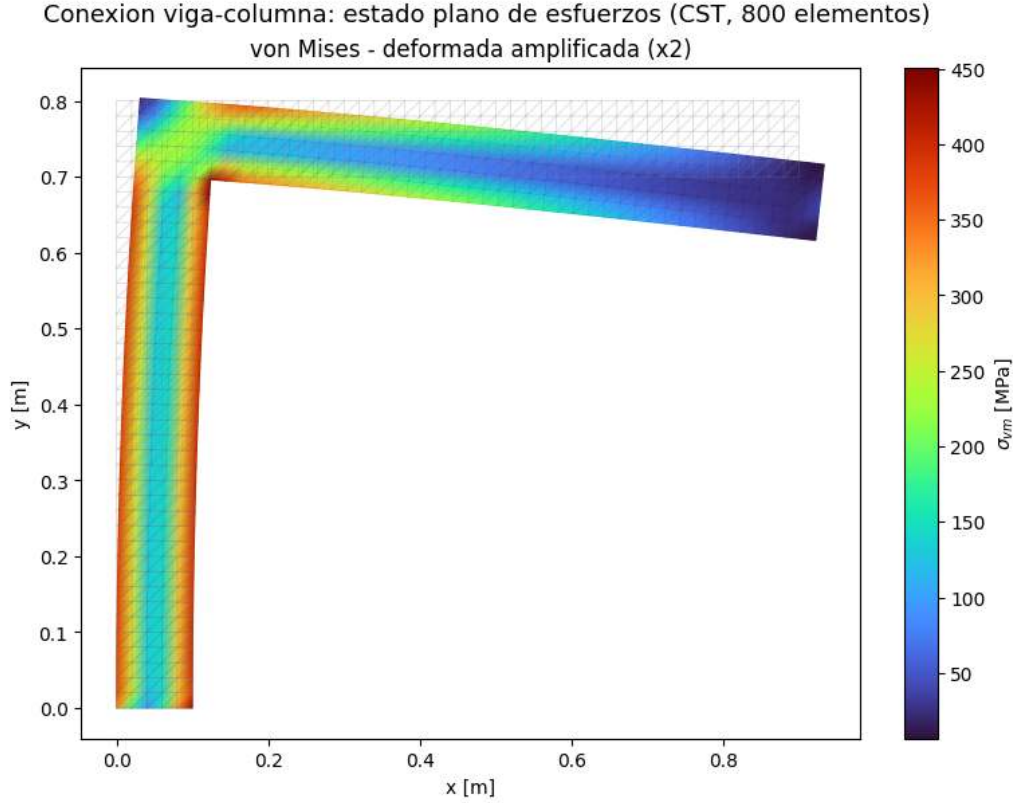


Figura 11: Mapa de esfuerzo equivalente de von Mises en una conexión viga–columna bajo estado plano de esfuerzos. Se considera acero estructural con $E = 200$ GPa, $\nu = 0.30$ y espesor $t = 10$ mm; la columna tiene dimensiones 0.10×0.80 m, la viga 0.80×0.10 m y se aplica una carga puntual descendente $P = 10$ kN en el extremo libre. La simulación emplea una malla de elementos triangulares CST con 486 nodos y 800 elementos. El campo de esfuerzos se muestra sobre la geometría deformada amplificada, con la silueta original en gris como referencia. Se observa concentración de esfuerzos en la esquina interior del nudo, flexión de la viga en voladizo y transmisión de momento hacia la columna. El esfuerzo máximo nodal estimado es $\sigma_{vm}^{max} \approx 450$ MPa.

1.20.6. Verificación analítica de orden de magnitud

Antes de aceptar el mapa FEM conviene verificar que el orden de magnitud es coherente con un cálculo manual. Aproximando la conexión como un portico en L con la viga como cantiliver, el momento flector en la base de la columna es

$$M \approx P(L_b + w_c/2) = 10\,000 \times 0.85 = 8\,500 \text{ N} \cdot \text{m}. \quad (84)$$

El modulo de sección de la columna (rectangular delgada) es $W = tw_c^2/6 = 0.010 \cdot 0.10^2/6 \approx 1.67 \times 10^{-5} \text{ m}^3$. El esfuerzo de flexión estimado es

$$\sigma_b \approx \frac{M}{W} \approx \frac{8500}{1.67 \times 10^{-5}} \approx 510 \text{ MPa}, \quad (85)$$

que es consistente con el rango 450–590 MPa obtenido por FEM. Esta verificación analítica es lo primero que se debe hacer ante cualquier resultado numérico: si el FEM y la estimación analítica difieren en un orden de magnitud, hay un error de unidades, de carga o de geometría.

1.20.7. Interpretación estructural

1. **Concentración en la esquina interior.** La esquina interior del nudo (donde se une la cara superior de la columna con la cara inferior de la viga) es la zona más brillante del mapa. Es una *singularidad geométrica*: en teoría de elasticidad lineal, una esquina entrante produce esfuerzos que tienden a infinito al refinar la malla, salvo redondeo (chafán/filete). Esto explica que el valor máximo dependa del tamaño de malla.
2. **Distribución en la columna.** El gradiente de esfuerzos a lo largo de la columna corresponde a la flexión: tracción en la cara izquierda y compresión en la derecha (o viceversa según el signo del momento), con línea neutra interna.
3. **Distribución en la viga.** El momento flector decrece linealmente desde el nudo hacia el extremo libre, donde tiende a cero. Esto se refleja en el oscurecimiento progresivo del mapa hacia la derecha.
4. **Comparación con σ_y .** El acero estructural típico (ASTM A36) tiene $\sigma_y \approx 250$ MPa. Con $\sigma_{vm}^{máx} \approx 450$ MPa, la estructura habría entrado en plasticidad y el modelo lineal pierde validez. Para esta carga se requeriría un análisis elasto-plástico, un material de mayor límite elástico o rediseñar la conexión.

1.20.8. Conexión con investigación doctoral

Este análisis es el bloque base de tres líneas de investigación comunes en ingeniería estructural y mecánica: (i) optimización topológica del nudo para reducir picos de esfuerzo; (ii) análisis de fatiga bajo carga cíclica usando $\sigma_{vm}^{máx}$ como amplitud equivalente; (iii) validación experimental mediante extensometría (galgas) o correlación digital de imágenes (DIC) en una probeta a escala. La transferencia al Artículo 9 del temario (*below-the-hook lifting devices integrating FEA and physical proof testing*) es directa.

1.21. Discusion de los articulos cientificos como casos de investigacion real

Los diez articulos seleccionados se usaran como casos de lectura metodologica. En esta primera semana no se busca que el estudiante domine todos los detalles tecnicos de cada articulo, sino que identifique la estructura de modelado: problema, abstraccion, ecuacion o algoritmo, simulacion, prototipo, validacion y limitaciones.

Tabla 5: Lectura metodologica de los articulos para la Semana 1.

No.	Caso	Modelo central	Preguntas para discusion doctoral
1	RAG para tareas intensivas en conocimiento	Pipeline de recuperacion y generacion	?Que se considera dato, modelo y prototipo? ?Como se valida una respuesta generada?
2	FinGPT	Modelo de lenguaje especializado	?Que diferencia existe entre adaptar un modelo y validar conocimiento financiero?
3	PINNs	Residuo fisico en funcion de perdida	?Cuando una red neuronal puede considerarse modelo cientifico y no solo predictor estadistico?
4	Dano en OPGW por descarga	FEM y validacion experimental	?Que variables fisicas son necesarias para explicar dano real?
5	Temperatura en OPGW por corto-circuito	Acoplamiento electro-termico	?Que simplificaciones son admisibles para diseno y cuales no?
6	Control inteligente de trafico	Aprendizaje por refuerzo	?Como se valida una politica antes de implementarla en una ciudad real?
7	Dispositivo medico de deteccion	Prototipo con vision artificial	?Que diferencia hay entre exactitud algoritmica y utilidad clinica?
8	Bomba centrifuga	CFD y optimizacion geometrica	?Como se conectan simulacion, geometria, desempeno y validacion experimental?

No.	Caso	Modelo central	Preguntas para discusion doctoral
9	Dispositivo de izaje	FEA y prueba fisica	?Que evidencia convierte un diseno simulado en un prototipo certificable?
10	Modelo de negocio circular	Simulacion basada en agentes	?Como se valida un modelo cuando los agentes son humanos u organizaciones?

1.21.1. Matriz comun de analisis

Para cada articulo se debe completar la siguiente matriz:

Tabla 6: Matriz de analisis para articulos de simulacion y prototipado.

Elemento	Descripcion a extraer del articulo
Problema de investigacion	?Que fenomeno, necesidad o brecha se aborda?
Sistema real	?Cual es el objeto fisico, computacional, social o biologico?
Modelo fisico/conceptual	?Que mecanismos se consideran dominantes?
Modelo matematico/-computacional	?Que ecuaciones, algoritmos o reglas se usan?
Variables de estado	?Que magnitudes evolucionan o se estiman?
Parametros	?Que constantes, propiedades o hiperparametros controlan el modelo?
Condiciones iniciales/-frontera	?Que restricciones o entradas se imponen?
Metodo de simulacion	FEM, CFD, IA, agentes, Monte Carlo, etc.
Prototipo	?Es fisico, digital, experimental, clinico, computacional o hibrido?
Validacion	?Contra que evidencia se compara el modelo?
Limitaciones	?Que no puede concluirse a partir del estudio?
Transferencia doctoral	?Como puede inspirar una tesis o proyecto propio?

1.22. Ejercicios guiados para uso de IA generativa en simulación

1.22.1. Principio operativo

La IA generativa se usará como asistente de codificación, no como autoridad científica. El estudiante debe pedir código, revisar unidades, ejecutar, modificar parámetros y detectar errores conceptuales. El entregable no será únicamente el código generado, sino la interpretación del modelo.

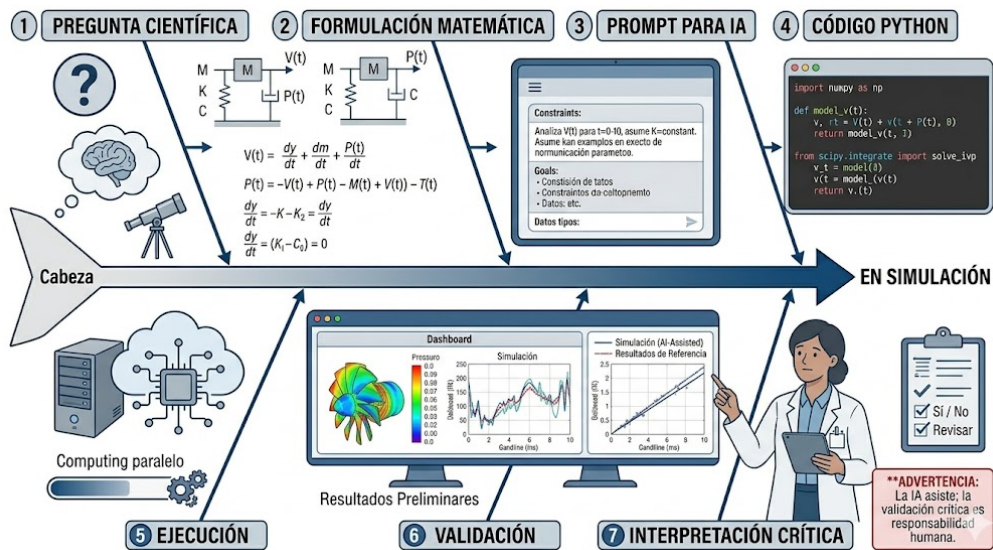


Figura 12: Flujo de trabajo para integración de IA generativa en simulación científica reproducible.

1.22.2. Ejercicio IA-1: simulación de difusión térmica

Objetivo. Usar IA para generar un código de simulación de conducción transitoria 1D y verificar estabilidad numérica.

Genera un script en Python para resolver la ecuación de calor unidimensional $\partial T / \partial t = \alpha \partial^2 T / \partial x^2$ en una barra de longitud $L = 1$ m. Usa diferencias finitas explícitas, condición inicial gaussiana, fronteras de Dirichlet $T(0, t) = T(L, t) = 25^\circ\text{C}$, $\alpha = 10^{-4} \text{ m}^2/\text{s}$, $N = 151$ nodos y un parámetro de estabilidad $r = \alpha \Delta t / \Delta x^2 = 0.45$. El código debe graficar perfiles de temperatura en cinco tiempos, imprimir Δx , Δt y verificar que $r \leq 0.5$. Incluye comentarios en español y unidades.

1.22.3. Ejercicio IA-2: masa–resorte–amortiguador

Genera un script en Python para simular un sistema masa–resorte–amortiguador descrito por $m\ddot{x} + c\dot{x} + kx = 0$. Usa $m = 1$ kg, $k = 100$ N/m y tres razones de amortiguamiento $\zeta = 0.05, 0.2$ y 0.8 . Calcula $c = 2\zeta\sqrt{km}$. Integra el sistema usando `scipy.integrate.solve_ivp`, grafica $x(t)$ y la energía mecánica $E(t) = 0.5mv^2 + 0.5kx^2$. Explica en comentarios como cambia la disipación de energía.

1.22.4. Ejercicio IA-3: modelo concentrado de calentamiento OPGW

Genera un script en Python para simular el calentamiento de un cable mediante el modelo concentrado $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Usa tres corrientes: 200 A, 400 A y 600 A. Define valores razonables para m, c_p, R, h, A_s y T_∞ , indicándolos en comentarios como parámetros de ejemplo no normativos. Integra durante 300 s, grafica $T(t)$, calcula temperatura máxima y marca una línea horizontal de umbral crítico a 80 °C.

1.22.5. Ejercicio IA-4: modelo basado en agentes

Genera un modelo basado en agentes en Python para simular la adopción de una tecnología de reciclaje. Usa 200 agentes, estado binario de adopción, función logística de probabilidad de adopción y un término de influencia social proporcional a la fracción global de adopción. Ejecuta tres escenarios de costo percibido: bajo, medio y alto. Grafica la fracción de adopción contra el tiempo y comenta como afecta el costo a la dinámica.

1.22.6. Ejercicio IA-5: conducción 2D en placa de intercambiador

Objetivo. Reproducir el Ejemplo 7 modificando parámetros para evaluar sensibilidad.

Genera un script en Python para resolver la ecuación de calor 2D $\partial T / \partial t = \alpha \nabla^2 T$ en una placa rectangular de 0.20×0.10 m con diferencias finitas explícitas FTCS y verificación del criterio $r_x + r_y \leq 1/2$. Condiciones Dirichlet: $T = 200$ °C en $x = 0$ y $y = L_y$; $T = 20$ °C en $x = L_x$ y $y = 0$. Condición inicial uniforme a 25 °C. Material: acero $k = 50$ W/(mK), $\rho = 7850$ kg/m³, $c_p = 480$ J/(kgK). Captura 5 snapshots:

$t = 0$, tres intermedios y estado estable definido por $\max |\Delta T| < 10^{-4} \text{ }^{\circ}\text{C}$. Grafica cada snapshot con `pcolormesh` y superpone isoterma. Imprime $\Delta x, \Delta y, \Delta t, r_x, r_y$ y el tiempo total al estado estable.

1.22.7. Ejercicio IA-6: von Mises en conexion viga-columna

Objetivo. Reproducir el Ejemplo 8 con FEM CST y discutir sensibilidad a la malla y a la carga.

Genera un script en Python que implemente desde cero un FEM 2D con elementos CST (Constant Strain Triangle) para una conexion viga-columna tipo L: columna $0.10 \times 0.80 \text{ m}$, viga $0.80 \times 0.10 \text{ m}$, acero $E = 200 \text{ GPa}$, $\nu = 0.30$, espesor $t = 10 \text{ mm}$. Empotra la base de la columna y aplica una carga puntual de 10 kN hacia abajo en el extremo libre de la viga. Calcula desplazamientos nodales, esfuerzos por elemento (estado plano de esfuerzos) y el campo de von Mises. Promedia von Mises a nodos (ponderado por area) y grafica el mapa sobre la geometria sin deformar y sobre la deformada amplificada. Verifica el orden de magnitud con la estimacion analitica $\sigma \approx M/W$. Repite con malla mas fina y discute la convergencia.

1.23. Actividad de refuerzo en clase: 30–40 minutos

1. Nombre de la actividad

Del artículo al prototipo: disección metodológica y simulación mínima asistida por IA.

2. Duración

Duración total estimada: **35 minutos**, distribuidos en selección del artículo (5 min), identificación de modelo (8 min), generación y ejecución (12 min), interpretación (5 min) y cierre grupal (5 min).

3. Organización

Los estudiantes trabajan en equipos de dos o tres integrantes. Cada equipo selecciona uno de los diez artículos y construye una versión mínima del modelo si el original es demasiado complejo para implementarse en 12 minutos.

4. Producto entregable

Cada equipo entrega una hoja breve con: artículo, fenómeno real, modelo físico/conceptual, ecuación o regla mínima, variables, parámetros, hipótesis, prompt usado, gráfica obtenida, interpretación técnica de máximo 120 palabras y una limitación explícita del modelo.

5. Rúbrica rápida

Tabla 7: Rúbrica de evaluación rápida para la actividad de 30–40 minutos.

Criterio	Excelente	Suficiente	Insuficiente
Identificación del modelo	Distingue fenómeno, modelo físico, modelo matemático y simulación.	Identifica parcialmente los elementos.	Confunde fenómeno, modelo y herramienta.
Variables y parámetros	Declara variables, parámetros y unidades.	Declara variables sin unidades completas.	Omite variables o usa términos ambiguos.
Uso de IA	Prompt claro, específico y verificable.	Prompt funcional pero incompleto.	Prompt genérico o no verificable.
Ejecución de simulación	Código ejecutado y resultado interpretado.	Código ejecutado con interpretación débil.	No hay ejecución o no hay resultado.
Juicio crítico	Declara limitaciones y validez del modelo.	Declara limitaciones superficiales.	No reconoce limitaciones.

1.24. Ejercicios para tarea

1.24.1. Ejercicio 1: clasificacion de modelos

Seleccione tres articulos de la tabla de casos. Para cada articulo, complete una tabla con cuatro columnas: modelo fisico, modelo matematico/computacional, simulacion y prototipo. En la columna de validacion indique que evidencia se empleo para evaluar el modelo.

1.24.2. Ejercicio 2: derivacion de ecuacion gobernante

A partir de un balance de energia, derive la ecuacion concentrada de calentamiento por efecto Joule $mc_p dT/dt = I^2 R - hA_s(T - T_\infty)$. Indique las hipotesis necesarias para que esta expresion sea valida y explique bajo que condicion deberia reemplazarse por un modelo distribuido.

1.24.3. Ejercicio 3: estabilidad numerica

Ejecute la simulacion de conduccion 1D con $r = 0.25$, $r = 0.45$ y $r = 0.60$. Compare los resultados y responda: ¿que caso es estable?, ¿que caso muestra oscilaciones o divergencia?, ¿por que el criterio $r \leq 0.5$ no es una recomendacion estetica, sino una condicion matematica?

1.24.4. Ejercicio 4: incertidumbre

Suponga que la temperatura final de un cable se estima mediante

$$T_f = T_\infty + \frac{I^2 R}{hA_s} (1 - e^{-hA_s t / (mc_p)}) . \quad (86)$$

Considere incertidumbre en I , R y h . Escriba el Jacobiano $J = [\partial T_f / \partial I, \partial T_f / \partial R, \partial T_f / \partial h]$ y explique como se propagaria la incertidumbre si se conoce la matriz de covarianza de entrada.

1.24.5. Ejercicio 5: convergencia de Monte Carlo

Ejecute el codigo Monte Carlo para π con $N = 10^3$, 10^4 , 10^5 y 10^6 muestras. Repita cada caso 30 veces variando la semilla y calcule la desviacion estandar empirica de $\hat{\pi}_N$. Verifique que $\sigma(\hat{\pi}_N) \propto N^{-1/2}$ ajustando una recta en escala log-log. Discuta el costo computacional necesario para obtener tres cifras decimales correctas en π .

1.24.6. Ejercicio 6: estado estable en placa 2D

Modifique el Ejemplo 7 sustituyendo la condición Dirichlet en $x = L_x$ por una condición de Robin con coeficiente convectivo $h = 200 \text{ W}/(\text{m}^2\text{K})$ y $T_\infty = 20^\circ\text{C}$. ¿Cómo cambia el campo de estado estable? ¿Cómo cambia el tiempo característico para alcanzarlo? Justifique con $Bi = hL_x/k$.

1.24.7. Ejercicio 7: sensibilidad estructural a la malla

Ejecute el Ejemplo 8 con tamaño de elemento $h_1 = 0.04 \text{ m}$, $h_2 = 0.02 \text{ m}$, $h_3 = 0.01 \text{ m}$. Reporte $\sigma_{vm}^{\text{máx}}$ para cada malla, estime el orden de convergencia observado p_{obs} y aplique extrapolación de Richardson. Discuta por qué $\sigma_{vm}^{\text{máx}}$ no converge en la esquina entrante y cómo se mitiga (filete, refinamiento adaptativo, regularización).

1.24.8. Ejercicio 8: prompt doctoral

Redacte un prompt de IA para generar una simulación relacionada con su propio tema de tesis. El prompt debe incluir: fenómeno, ecuación o algoritmo, parámetros, unidades, condiciones de frontera o reglas de decisión, método numérico, gráfica esperada y criterio de validación. No se aceptan prompts genéricos del tipo “haz una simulación”, porque eso no es ingeniería, es lanzar una moneda con teclado.

1.25. Cierre conceptual

El modelado científico no consiste en convertir cualquier problema en código. Consiste en establecer una correspondencia defendible entre un fenómeno y una representación formal. Una simulación útil debe responder una pregunta concreta, declarar sus hipótesis, especificar sus parámetros, controlar sus errores y validarse contra evidencia.

En esta primera semana se establecen los criterios que se usarán durante todo el curso:

1. Todo modelo debe declarar su fenómeno de referencia.
2. Toda simulación debe derivarse de un modelo explícito.
3. Todo prototipo debe tener una función de validación o exploración.
4. Toda gráfica debe responder una pregunta técnica.
5. Todo código debe ser reproducible.
6. Toda conclusión debe reconocer su dominio de validez.

La transición de fenómeno a modelo, y de modelo a prototipo, será el eje de las siguientes semanas. La Semana 2 se dedicará a herramientas computacionales científicas, con énfasis en scripts reproducibles, visualización y automatización de cálculos. La Semana 3 profundizará en discretización, estabilidad, convergencia y métodos numéricos fundamentales. A partir de ahí, cada tema del curso añadirá una capa técnica sobre la base metodológica establecida en esta sesión.

Apéndice Semana 1. Referencias de los casos discutidos

- A1. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S. y Kiela, D. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. DOI: <https://doi.org/10.48550/arXiv.2005.11401>.
- A2. Liu, X., Wang, G., Yang, H. y Zha, D. (2023). *FinGPT: Democratizing Internet-scale Data for Financial Large Language Models*. DOI: <https://doi.org/10.48550/arXiv.2306.06031>.
- A3. Raissi, M., Perdikaris, P. y Karniadakis, G. E. (2019). *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*. *Journal of Computational Physics*. DOI: <https://doi.org/10.1016/j.jcp.2018.10.045>.
- A4. Sun, J., Wang, Y., Liu, Z., Zhang, X. y otros. (2020). *Experimental and numerical analysis of damage characteristics to OPGW strands under first lightning strike and continuous current*. *Electric Power Systems Research*. DOI: <https://doi.org/10.1016/j.epsr.2020.106515>.
- A5. Ashfaq, A., et al. (2025). *Comprehensive analysis of temperature distribution in OPGW cable considering ramifications of short circuit current*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-025-09266-7>.
- A6. Wei, H., Zheng, G., Yao, H. y Li, Z. (2018). *IntelliLight: A Reinforcement Learning Approach for Intelligent Traffic Light Control*. DOI: <https://doi.org/10.1145/3219819.3220096>.
- A7. Birur, N. P., et al. (2022). *Field validation of deep learning based Point-of-Care device for early detection of oral malignant and potentially malignant disorders*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-022-18249-x>.
- A8. Yazdanpanah-Ardakani, K., et al. (2024). *Optimization of a centrifugal blood pump designed using artificial intelligence*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-024-57019-9>.
- A9. Habib, M. y Shamso, M. (2025). *A validated workflow for the design and certification of below-the-hook lifting devices integrating finite element analysis and physical proof testing*. *Scientific Reports*. DOI: <https://doi.org/10.1038/s41598-025-31175-y>.
- A10. Lange, K. P. H., Korevaar, G., Oskam, I. F. y Herder, P. M. (2021). *Agent-based modelling and simulation for circular business model experimentation*. *Resources, Conservation and Recycling Advances*. DOI: <https://doi.org/10.1016/j.rcradv.2021.200055>.